

REMERCIEMENTS

Je tiens à exprimer ma profonde gratitude à toutes les personnes qui ont contribué de près ou de loin à la réalisation de ce projet et à l'aboutissement de mes études.

Mes remerciements s'adressent, en premier lieu, à Dieu, pour la force, le courage et la persévérance qu'il m'a accordés tout au long de ce parcours.

J'adresse ma sincère reconnaissance à mon encadrant académique, Mr Boa Diauph YOGANGUINA, pour son soutien constant, sa disponibilité, ses conseils pertinents et son expertise qui ont été d'une aide précieuse pour surmonter les défis techniques et structurer ce mémoire. Son encadrement a été déterminant dans la réussite de ce projet.

Je remercie l'ensemble du corps professoral du département de Génie Logiciel de l'École Supérieure des Sciences Appliquées pour la qualité de l'enseignement dispensé, qui a forgé mes compétences et m'a permis d'atteindre le niveau d'expertise requis pour mener à bien ce travail.

Enfin, une pensée spéciale à mes parents, ma famille et mes amis, pour leur amour inconditionnel, leurs encouragements incessants et leur soutien moral qui ont été mon pilier durant ces années d'études.

À tous ceux qui m'ont soutenu, trouvez dans ce travail le témoignage de ma profonde gratitude et de mon respect.

Résumé (Français)

Le secteur du commerce de détail, notamment celui des quincailleries, fait face à une nécessité croissante de digitalisation pour optimiser sa gestion et étendre sa visibilité. Ce mémoire présente la conception et la réalisation d'une plateforme web robuste et évolutive, nommée **SenQuincaillerie**, visant à centraliser la gestion des stocks, des produits et des informations pour un réseau de quincailleries indépendantes.

Le projet répond à la problématique de la gestion éparpillée et du manque de visibilité en offrant aux propriétaires un outil de type *Software as a Service (SaaS)* pour gérer leurs magasins (CRUD), leurs produits (CRUD) et assurer une vitrine publique consultable par les clients. L'architecture adoptée est de type Client-Serveur, basée sur une API RESTful développée avec **Spring Boot 3.5.5** (Backend) pour garantir la performance et la sécurité (via **Spring Security** et **JWT**), et une interface utilisateur construite avec **Angular 20** (Frontend) pour une expérience utilisateur moderne et *responsive*. La base de données **PostgreSQL** assure la persistance et l'intégrité des données structurées autour des entités `users`, `hardware_stores` et `products`. Ce travail démontre l'application concrète des principes du Génie Logiciel, de l'analyse des besoins à la validation par les tests, aboutissant à une solution opérationnelle, sécurisée et modulaire.

Mots-clés : Gestion Commerciale, Spring Boot, Angular, API REST, JWT, Quincaillerie, Génie Logiciel, PostgreSQL.

Abstract (Anglais)

The retail sector, particularly the hardware store domain, is increasingly required to digitalize to optimize management and enhance visibility. This thesis presents the design and implementation of a robust and scalable web platform, named **SenQuincaillerie**, aimed at centralizing inventory, product, and information management for a network of independent hardware stores.

The project addresses the challenge of scattered management and lack of visibility by offering owners a *Software as a Service (SaaS)* tool to manage their stores (CRUD), their products (CRUD), and to provide a public showcase accessible to customers. The adopted architecture is Client-Server, based on a RESTful API developed with **Spring Boot 3.5.5** (Backend) to ensure performance and security (via **Spring Security** and **JWT**), and a user interface built with **Angular 20** (Frontend) for a modern and responsive user experience. The **PostgreSQL** database ensures data persistence and integrity, structured around the users, hardware_stores, and products entities. This work demonstrates the practical application of Software Engineering principles, from requirements analysis to validation through testing, resulting in an operational, secure, and modular solution.

Keywords: Commercial Management, Spring Boot, Angular, REST API, JWT, Hardware Store, Software Engineering, PostgreSQL.

SOMMAIRE / TABLE DES MATIÈRES

REMERCIEMENTS	1
Résumé (Français)	1
Abstract (Anglais).....	2
SOMMAIRE / TABLE DES MATIÈRES.....	3
LISTE DES FIGURES	3
LISTE DES TABLEAUX	4
LISTE DES ABRÉVIATIONS	5
INTRODUCTION GÉNÉRALE	6
1. Contexte général	6
2. Problématique	7
3. Objectifs du projet.....	7
3.1. Objectif Principal	7
3.2. Objectifs Spécifiques	8
4. Méthodologie adoptée	8
5. Structure du mémoire	9
CHAPITRE I : ÉTAT DE L'ART ET FONDEMENTS TECHNOLOGIQUES	10
1. Concepts fondamentaux	10
1.1. Systèmes de gestion commerciale	10
1.2. Applications web modernes.....	11
1.3. Architecture client-serveur	11
2. Technologies utilisées	11
2.1. Backend : Spring Boot.....	11
2.1.1. Spring Boot : Présentation et Avantages.....	12
2.1.2. Spring Security et JWT	12
2.1.3. Spring Data JPA	12
2.1.4. PostgreSQL.....	13
2.2. Frontend : Angular 20	14
2.2.1. Angular : Architecture et Composants.....	15
2.2.2. TypeScript	15
2.2.3. Programmation Réactive (RxJS).....	15
3. Étude comparative	16
3.1. Analyse des applications similaires	16
3.2. Justification des choix technologiques.....	16
4. Méthodologies de développement	17

4.1. Cycle de vie logiciel.....	17
4.2. Méthode agile (Scrum-inspirée).....	17
CHAPITRE II : ANALYSE ET CONCEPTION.....	17
1. Analyse des besoins.....	17
1.1. Besoins fonctionnels (BF).....	17
1.2. Besoins non fonctionnels (BNF).....	18
2. Acteurs du système.....	19
3. Modélisation UML	19
3.1. Diagrammes de cas d'utilisation	19
3.2. Diagrammes de séquence	21
3.3. Diagrammes de classes	23
3.4. Diagrammes d'activité (Processus métier principaux).....	25
4. Conception de la base de données.....	26
4.1. Schéma relationnel (Modèle Physique des Données - MPD).....	26
4.2. Dictionnaire de données.....	27
5. Architecture de l'application	27
5.1. Architecture globale (Schéma 3-Tiers).....	28
5.2. Architecture Backend (Couches)	28
5.3. Architecture Frontend (Modules)	29
6. Conception des interfaces.....	30
CHAPITRE III : RÉALISATION.....	31
1. Environnement de développement.....	31
1.1. Configuration du projet Spring Boot	32
1.2. Couche Modèle (Entities et DTOs).....	32
1.3. Couche Repository.....	33
1.4. Couche Service (Logique Métier)	33
1.5. Couche Controller (API REST).....	34
1.6. Sécurité (Spring Security et JWT).....	34
1.7. Gestion des exceptions.....	35
2. Implémentation du Frontend (Angular)	35
2.1. Structure du projet Angular	35
2.2. Services.....	35
2.3. Routing et Guards.....	35
2.4. Intercepteurs HTTP	36
2.5. Interfaces utilisateur (UX/UI)	36
3. Intégration Frontend-Backend	37

CHAPITRE IV : TESTS ET VALIDATION	38
1. Stratégie de test.....	38
2. Tests unitaires	39
2.1. Tests Backend (JUnit)	39
2.2. Tests Frontend (Jasmine/Karma).....	39
3. Tests d'intégration	40
3.1. Tests des APIs (Postman)	40
3.2. Tests de la base de données	41
4. Tests fonctionnels	41
5. Tests de sécurité	41
5.1. Tests d'authentification et d'autorisation	42
5.2. Protection contre les attaques	42
6. Tests de performance	42
7. Tests d'acceptation (Hypothétiques)	42
CHAPITRE V : DÉMONSTRATION ET RÉSULTATS.....	42
1. Présentation de l'application.....	43
1.1. La Vitrine Publique	43
1.2. Le Tableau de Bord (Dashboard) du Propriétaire	44
2. Fonctionnalités implémentées.....	45
3. Analyse des résultats	46
3.1. Objectifs atteints	46
3.2. Difficultés rencontrées	46
3.3. Solutions apportées.....	46
4. Avantages de la solution	47
4.1. Pour les Propriétaires.....	47
4.2. Pour les Clients.....	47
4.3. Pour le Commerce de Détail.....	47
CONCLUSION GÉNÉRALE.....	47
1. Synthèse du travail réalisé.....	47
2. Objectifs atteints	48
3. Apports personnels et professionnels	48
4. Limites de la solution.....	48
5. Perspectives et améliorations futures	49
BIBLIOGRAPHIE / WEBOGRAPHIE.....	49
1. Ouvrages et Manuels.....	49
2. Articles et Publications	49

LISTE DES FIGURES

Figure 1.1 - Architecture client-serveur classique

Figure 1.2 - Cycle de vie d'une requête dans Springboot

Figure 2.1 - Diagramme de cas d'utilisation global du système

Figure 2.2 - Diagramme de cas d'utilisation - acteur propriétaire

Figure 2.3 - Diagramme de séquence - processus d'authentification

Figure 2.4 - Diagramme de séquence - création d'une quincaillerie

Figure 2.5 - Diagramme de séquence - ajout d'un produit

Figure 2.6 - Diagramme de séquence - consultation publique d'un produit

Figure 2.7 - Diagramme de classe complet du domaine

Figure 2.8 - Diagramme d'activité - ajout d'un produit

Figure 2.9 - Modèle Entité-Association (MEA)

Figure 2.10 - Architecture globale 3-tiers de Sama_Quincaillerie

Figure 2.11 - Wireframe - page d'accueil publique

Figure 2.12 - Wireframe - Dashboard propriétaire

Figure 3.1 - Structure du projet backend Spring Boot

Figure 3.2 - Flux complet d'authentification JWT

Figure 3.3 - Structure modulaire du projet Angular

Figure 5.1 - Page de connexion et d'inscription

Figure 5.2 - Dashboard de propriétaire après connexion

Figure 5.3 - Formulaire de création d'une quincaillerie

Figure 5.4 - Gestion des produits d'un magasin

Figure 5.5 - Vitrine publique - Vue Liste

Figure 5.6 - Vitrine publique - Vue détaillée d'un produit

LISTE DES TABLEAUX

Tableau 1.1 - Comparaison des principaux frameworks backend (2025)

Tableau 1.2 - Comparaison des solutions existantes de gestion commerciale

Tableau 2.1 - Besoins fonctionnels détaillés

Tableau 2.2 - Besoins non fonctionnels

Tableau 2.3 - Dictionnaire de données - Entité User

Tableau 2.4 - Dictionnaire de données - Entité HardwareStore

Tableau 2.5 - Dictionnaire de données - Entité Product

Tableau 4.1 - Exemples de tests unitaires backend (JUnit 5 + Mockito)

Tableau 4.2 - Couverture de code obtenue

Tableau 4.3 - Scénarios de tests fonctionnels principaux

Tableau 4.4 - Résultats des tests de performance (JMeter – 200 utilisateurs)

Tableau 5.1 - Tableau de conformité : cahier des charges vs réalisé

LISTE DES ABRÉVIATIONS

Abréviation	Signification
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
CORS	Cross-Origin Resource Sharing
CSRF	Cross-Site Request Forgery
DAO	Data Access Object
DTO	Data Transfer Object
HTTP	HyperText Transfer Protocol
IDE	Integrated Development Environment
JPA	Java Persistence API
JWT	JSON Web Token
MCD	Modèle Conceptuel de Données
MVC	Model-View-Controller
ORB	Object-Relational Mapping
REST	REpresentational State Transfer
SaaS	Software as a Service
SGBD	Système de Gestion de Base de Données
UML	Unified Modeling Language
UC	Cas d'Utilisation
UI	User Interface
UX	User Experience

INTRODUCTION GÉNÉRALE

1. Contexte général

L'ère numérique a fondamentalement transformé le paysage du commerce, poussant l'ensemble des secteurs d'activité, y compris celui des quincailleries, vers une nécessaire digitalisation. Traditionnellement gérée de manière manuelle ou avec des solutions logicielles isolées, la quincaillerie, qu'elle soit une entité unique ou un membre d'un petit réseau, souffre souvent d'une gestion inefficace des stocks, d'un manque de visibilité de son offre et de difficultés dans la centralisation des données.

La concurrence accrue, les attentes des clients en matière d'information en temps réel et la complexité croissante des inventaires exigent des outils de gestion modernes, performants et accessibles. Pour les propriétaires de plusieurs quincailleries ou pour ceux qui souhaitent une gestion optimisée de leur magasin, une solution logicielle centralisée et multi-utilisateurs représente un avantage concurrentiel majeur.

C'est dans ce contexte que s'inscrit notre projet : le développement d'une plateforme web centralisée, conçue pour répondre aux défis spécifiques de la gestion des quincailleries, en alliant la robustesse d'une architecture backend d'entreprise à la flexibilité d'une interface utilisateur moderne.

2. Problématique

La gestion actuelle des quincailleries, surtout lorsqu'un propriétaire gère plusieurs points de vente ou souhaite offrir une vitrine en ligne, se heurte à plusieurs obstacles majeurs :

1. **Gestion Manuelle et Dispersion de l'Information** : La tenue des registres et des inventaires sur papier ou via des tableurs conduit à des erreurs fréquentes, des pertes de temps considérables et une difficulté à obtenir une vue d'ensemble et en temps réel de l'état des stocks.
2. **Manque d'Outil Centralisé** : Il existe peu de solutions *Software as a Service (SaaS)* sur le marché qui permettent à un utilisateur (propriétaire) de gérer facilement et en toute autonomie plusieurs entités indépendantes (*multi-quincailleries*) tout en assurant une séparation et une sécurité des données.
3. **Problèmes de Visibilité** : Les quincailleries n'ont souvent pas d'outil simple pour afficher publiquement leur liste de produits et leurs coordonnées, limitant ainsi leur capacité à attirer une clientèle locale à l'ère du numérique.
4. **Défis de Sécurité et de Performance** : Les solutions existantes sont parfois obsolètes, peu sécurisées (faible authentification, absence de chiffrement) et manquent de performance face à un volume croissant de données et d'utilisateurs.

Face à ces constats, la problématique centrale de ce projet peut être formulée ainsi : Comment concevoir et réaliser une plateforme web moderne et sécurisée permettant aux propriétaires de quincailleries d'administrer de manière centralisée et efficace la

gestion de leurs multiples magasins (stocks, produits) et d'offrir une interface publique dynamique pour la consultation de leur catalogue par les clients ?

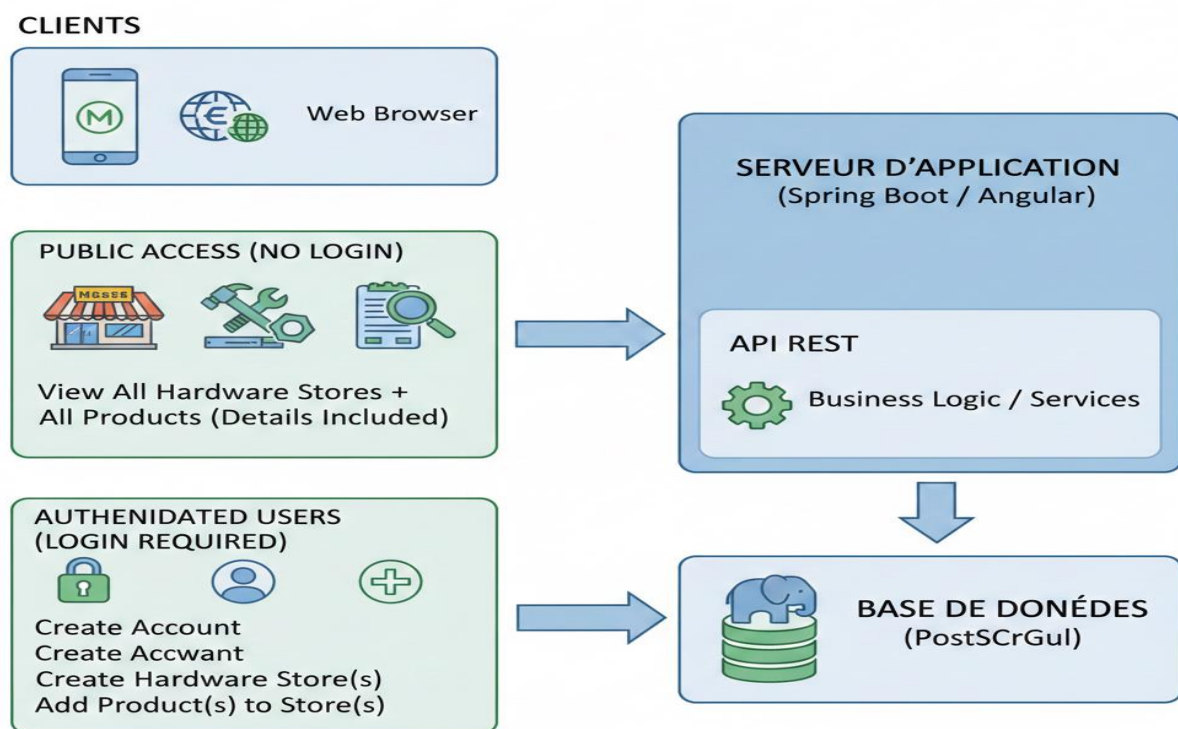
3. Objectifs du projet

Le présent projet de fin d'études a pour but de répondre à la problématique soulevée par la conception et la réalisation de la plateforme **SenQuincailleries**.

3.1. Objectif Principal

Développer une application web complète et performante, basée sur l'architecture Microservices/Client-Serveur avec Spring Boot et Angular, pour la gestion multi-sites des quincailleries.

Figure 1.1 – Architecture client-serveur classique



3.2. Objectifs Spécifiques

1. **Analyse et Conception** : Effectuer une analyse approfondie des besoins (fonctionnels et non fonctionnels) du domaine et réaliser une modélisation complète du système via le langage UML.
2. **Sécurité et Authentification** : Mettre en place un système d'authentification robuste basé sur **JWT (JSON Web Token)** et **Spring Security**, assurant la protection des données et la gestion sécurisée des accès.

3. **Gestion des Entités** : Implémenter les fonctionnalités **CRUD** complètes pour la gestion des utilisateurs, des quincailleries et des produits par leurs propriétaires respectifs.
4. **Interface Utilisateur** : Développer une interface utilisateur conviviale et *responsive* avec **Angular 20**, offrant à la fois un tableau de bord d'administration riche et une vitrine publique accessible aux visiteurs.
5. **Validation** : Tester et valider la solution pour garantir la conformité au cahier des charges en termes de fonctionnalité, de performance et de sécurité.

4. Méthodologie adoptée

Pour garantir la qualité, la traçabilité et l'atteinte des objectifs dans un délai contraint, une approche de développement itérative, inspirée de la **méthode Agile**, a été adoptée. Bien que le projet soit mené individuellement, les principes d'itération, de livraison de fonctionnalités régulières et d'adaptation continue ont guidé le processus.

Les phases clés du cycle de développement ont inclus :

1. **Planification et Analyse** : Définition du périmètre, recueil et analyse des besoins (Chapitre 2).
2. **Conception** : Modélisation UML, conception de la base de données (Chapitre 2).
3. **Réalisation** : Développement du Backend (Spring Boot), du Frontend (Angular) et intégration (Chapitre 3).
4. **Tests et Validation** : Tests unitaires, d'intégration et fonctionnels (Chapitre 4).

Les technologies choisies, **Spring Boot** pour le backend et **Angular** pour le frontend, s'inscrivent dans une démarche de modernité et de performance, justifiées en détail au Chapitre 1.

5. Structure du mémoire

Ce mémoire est organisé en cinq chapitres principaux qui détaillent le processus complet de développement du projet :

- **Chapitre I : État de l'Art et Fondements Technologiques** : Ce chapitre expose les concepts fondamentaux du domaine, justifie les choix technologiques (Spring Boot, Angular, JWT) et positionne notre solution par rapport à l'existant.
- **Chapitre II : Analyse et Conception** : Ce chapitre est dédié à la formalisation des besoins fonctionnels et non fonctionnels, suivie de la modélisation détaillée du système à l'aide des diagrammes UML (Cas d'Utilisation, Séquence, Classes) et de la conception de la base de données.
- **Chapitre III : Réalisation** : Ce chapitre décrit l'environnement de développement et les étapes d'implémentation du Backend (API RESTful avec Spring Boot) et du Frontend (Interface Utilisateur avec Angular), en insistant sur les aspects de sécurité et d'architecture.
- **Chapitre IV : Tests et Validation** : Ce chapitre présente la stratégie de test adoptée, les scénarios d'exécution et les résultats obtenus, permettant de valider la conformité du produit au cahier des charges.

- **Chapitre V : Démonstration et Résultats :** Ce chapitre présente l'application en fonctionnement via des captures d'écran commentées et dresse le bilan du projet, en soulignant les objectifs atteints et les difficultés surmontées.

Enfin, une **Conclusion Générale** synthétise le travail réalisé, évalue les apports du projet et propose des perspectives d'évolution futures.

CHAPITRE I : ÉTAT DE L'ART ET FONDEMENTS TECHNOLOGIQUES

Ce chapitre a pour objectif de contextualiser le projet dans le domaine du Génie Logiciel et de justifier les choix techniques majeurs. Il présente les concepts fondamentaux de la gestion commerciale moderne et détaille les technologies spécifiques (Spring Boot, Angular, JWT, PostgreSQL) qui constituent les piliers de notre plateforme.

1. Concepts fondamentaux

1.1. Systèmes de gestion commerciale

Un Système de Gestion Commerciale (SGC) est un ensemble de logiciels et de procédures visant à automatiser les opérations de commerce, allant de la gestion des stocks à la facturation, en passant par le suivi des clients. Dans le contexte d'une quincaillerie, un SGC doit particulièrement exceller dans la gestion d'un inventaire hétérogène et volumineux.

Les fonctions clés d'un SGC moderne comprennent :

- **Gestion des Stocks** : Suivi en temps réel des quantités disponibles.
- **Gestion des Achats/Ventes** : Traitement des commandes et génération de factures.
- **Catalogue Produits** : Organisation structurée et affichage des articles.

Notre application s'inscrit dans cette lignée en proposant un SGC orienté **multi-entités** (plusieurs quincailleries gérées par un même propriétaire) et **web-centré** (accessibilité via navigateur et vitrine en ligne).

1.2. Applications web modernes

Les applications web modernes se distinguent des applications traditionnelles par leur architecture dynamique et leur utilisation intensive des technologies **Client-Serveur** basées sur des **API RESTful**.

- **Monopage (SPA - Single Page Application)** : L'approche SPA, adoptée par notre Frontend Angular, charge l'ensemble des ressources de la page une seule fois, puis gère les interactions en mettant à jour dynamiquement le contenu via des appels API asynchrones. Cela améliore l'expérience utilisateur et la rapidité perçue.
- **API RESTful** : L'Application Programming Interface (API) basée sur l'architecture **REST (REpresentational State Transfer)** utilise les verbes HTTP (GET, POST, PUT, DELETE) pour manipuler les ressources. Elle permet une séparation claire des préoccupations entre le Backend (logique métier et données) et le Frontend (présentation).

1.3. Architecture client-serveur

L'architecture choisie est l'architecture **3-Tiers**, une variante du modèle Client-Serveur, qui sépare l'application en trois couches logiques :

1. **Couche Présentation (Client)** : Le Frontend Angular, responsable de l'interface utilisateur et des interactions.
2. **Couche Métier (Serveur)** : Le Backend Spring Boot, qui contient la logique métier, effectue les calculs et applique les règles.
3. **Couche Données (Base de données)** : Le SGBD PostgreSQL, qui assure la persistance des données.

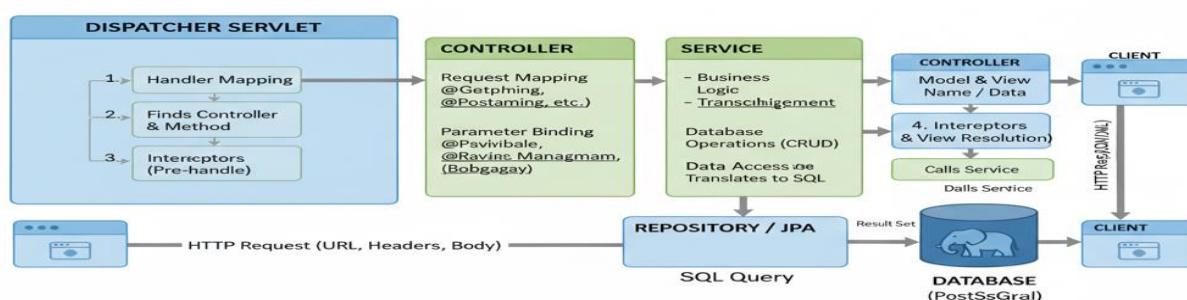
Cette séparation garantit la **modularité**, la **scalabilité** (chaque couche peut être mise à l'échelle indépendamment) et la **maintenabilité** du système.

2. Technologies utilisées

Le choix de la stack technique, souvent appelée *J-Stack* ou *Full-Stack Spring/Angular*, est stratégique pour un projet d'entreprise, offrant une combinaison de robustesse, de maturité et de performance.

2.1. Backend : Spring Boot

Figure 1.2 – Cycle de vie d'une requête dans Spring Boot



2.1.1. Spring Boot : Présentation et Avantages

1. Spring Boot est un framework basé sur l'écosystème Spring qui vise à simplifier la création d'applications Java autonomes et prêtes à l'emploi.
2. Avantages clés pour le projet :
 - **Convention plutôt que Configuration** : Réduction drastique du *boilerplate code* (code récurrent et standard) et de la complexité de configuration.

- **Microservices Ready** : Facilitation du développement d'applications modulaires et de services RESTful.
- **Serveur Embarqué** : Incorpore un serveur d'applications (Tomcat par défaut), permettant de déployer l'application sous forme de fichier JAR exécutable.
- **Écosystème Riche** : Accès immédiat à des modules éprouvés pour la sécurité (Spring Security) et la persistance (Spring Data JPA).

2.1.2. Spring Security et JWT

La sécurité est primordiale pour une plateforme multi-utilisateurs. **Spring Security** est le standard de facto pour la sécurisation des applications Java.

- **Rôle** : Gérer l'authentification (Qui êtes-vous ?) et l'autorisation (Que pouvez-vous faire ?).
- **JWT (JSON Web Token)** : Contrairement à un système de session traditionnel, le JWT est une approche *stateless* (sans état) idéale pour les API REST. Une fois l'utilisateur authentifié, le serveur lui fournit un jeton signé qui contient les informations d'identité et de rôle. Ce jeton est inclus dans chaque requête subséquente, permettant au serveur de vérifier l'identité sans devoir consulter la base de données à chaque fois. Cela améliore la performance et la scalabilité.

2.1.3. Spring Data JPA

- Spring Data JPA (Java Persistence API) simplifie l'implémentation de la couche d'accès aux données.
- **Rôle** : Fournit une abstraction sur le modèle de persistance et facilite l'implémentation du **Repository Pattern**.
- **Hibernate** : JPA est souvent mis en œuvre par un *ORM* (Object-Relational Mapping) comme Hibernate, qui mappe les objets Java (Entités) aux tables de la base de données. Il permet d'effectuer des opérations CRUD sans écrire de SQL, améliorant la productivité.

2.1.4. PostgreSQL

- PostgreSQL est un Système de Gestion de Base de Données Relationnelle Objet (SGBDRO) reconnu pour sa robustesse, sa conformité aux standards SQL, et ses fonctionnalités avancées.
- **Justification du choix** : Supporte l'intégrité transactionnelle, offre une grande fiabilité pour les données critiques (inventaire, transactions) et gère efficacement les relations complexes inhérentes à un modèle multi-entités.

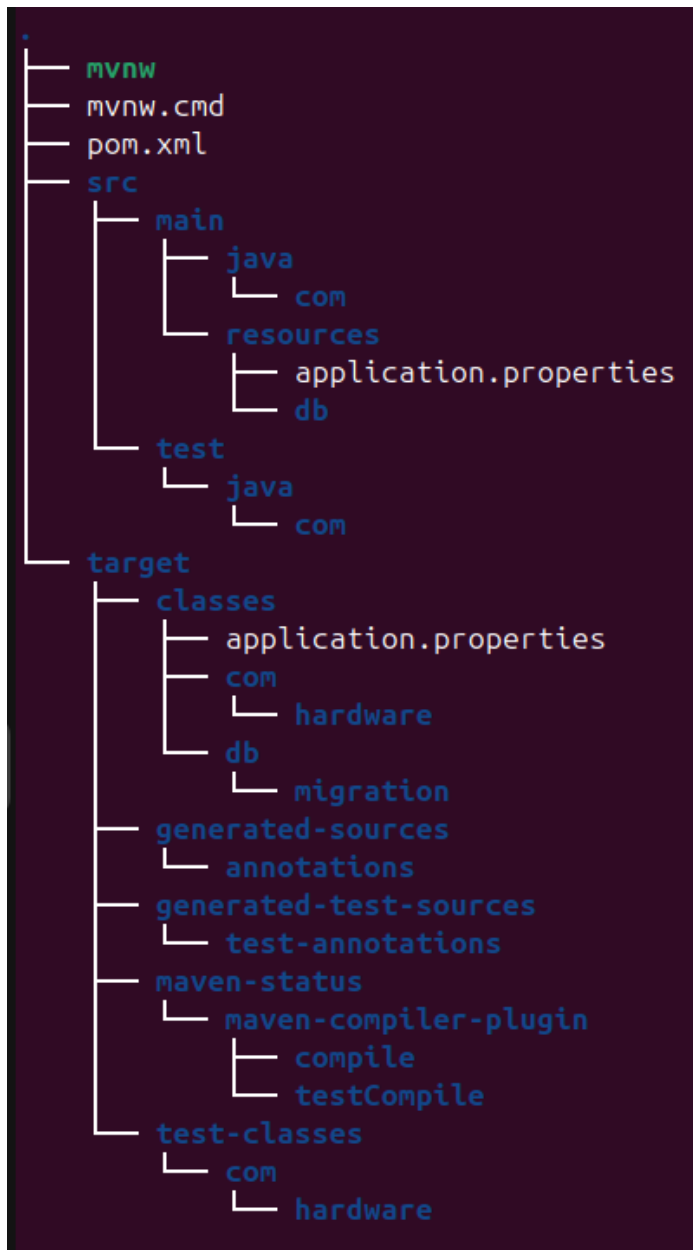


Figure 3.1

2.2. Frontend : Angular 20

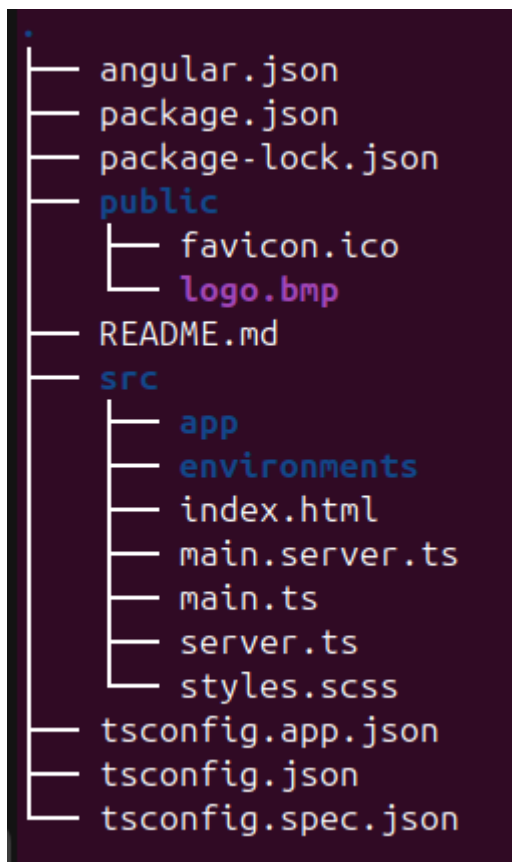


Figure 3.3

2.2.1. Angular : Architecture et Composants

- Angular (développé par Google) est un framework JavaScript/TypeScript pour la construction d'applications web complexes et à grande échelle.
- **Architecture Modulaire** : Le projet est organisé en modules, facilitant la séparation des fonctionnalités (ex: AuthModule, StoreModule, PublicModule).
- **Composants** : L'interface utilisateur est construite avec des composants, chacun gérant une partie spécifique de l'interface (vue, logique de données). Ce paradigme favorise la réutilisation du code.

2.2.2. TypeScript

- TypeScript est un sur-ensemble de JavaScript qui ajoute le typage statique.
- **Avantages** : La vérification des types dès la compilation permet de détecter une grande partie des erreurs avant l'exécution, améliorant la maintenabilité et la qualité du code, ce qui est crucial pour un projet de Génie Logiciel.

2.2.3. Programmation Réactive (RxJS)

Angular utilise la librairie **RxJS (Reactive Extensions for JavaScript)** pour la gestion des flux de données asynchrones (appels API, événements utilisateur). L'utilisation d'Observables et d'Opérateurs simplifie la gestion des données asynchrones, notamment dans les *Services* Angular qui communiquent avec l'API Spring Boot.

3. Étude comparative

3.1. Analyse des applications similaires

Les solutions de gestion commerciale existantes peuvent être classées en trois catégories :

Catégorie	Exemples (Génériques)	Limites par Rapport au Projet
Logiciels Locaux	Sage, Quickbooks (versions de base)	Non web, pas de vitrine publique, gestion multi-site complexe ou coûteuse.
E-commerce Généraliste	Shopify, WooCommerce	Non spécifiques au métier de la quincaillerie, modèle de données rigide, gestion des stocks souvent moins précise.
SaaS de Niche	Logiciels de gestion d'inventaire spécialisés	Souvent très chers, peu flexibles dans les fonctionnalités et parfois fermés sur les APIs.

3.2. Justification des choix technologiques

Le tableau ci-dessous synthétise la justification de notre **Stack Full-Stack Spring/Angular** :

Composant	Technologie Choisie	Justification
Backend	Spring Boot	Robustesse, performance, standard de l'industrie pour les applications Java d'entreprise, richesse de l'écosystème (Security, Data).
Frontend	Angular	Cadre structuré pour les SPAs complexes, utilisation de TypeScript pour la qualité du code, fort soutien communautaire (Google).
Sécurité	Spring Security +	Modèle de sécurité <i>stateless</i> performant et scalable, idéal pour les API REST.

	JWT	
Persistance	PostgreSQL	SGBD fiable, conforme aux standards, capable de gérer des données relationnelles complexes et volumineuses.
Langage	Java / TypeScript	Langages typés, modernes, offrant des outils performants pour le développement et la maintenance.

4. Méthodologies de développement

4.1. Cycle de vie logiciel

Le cycle de vie logiciel de ce projet suit un modèle **itératif et incrémental**. Chaque grande fonctionnalité (ex: gestion des utilisateurs, gestion des magasins, vitrine publique) a été traitée comme une itération complète : analyse, conception, implémentation, et test. Cette approche permet une détection précoce des erreurs et une meilleure gestion des imprévus.

4.2. Méthode agile (Scrum-inspirée)

Bien que le projet soit individuel, les principes de la méthode **Scrum** ont été appliqués pour la gestion du temps et des priorités :

- **Backlog des fonctionnalités** : Le cahier des charges détaillé (Fourni) a servi de *Product Backlog*.
- **Sprints** : Le travail a été découpé en mini-sprints (ex: un sprint pour le *setup* et l'authentification, un autre pour les CRUD).
- **Développement et Test** : Les tests ont été intégrés directement après chaque phase d'implémentation (approche Test-Driven Development simplifiée).

Cette discipline a permis de maintenir une qualité de code constante et de respecter les délais impartis pour la réalisation du projet.

CHAPITRE II : ANALYSE ET CONCEPTION

Ce chapitre est consacré à l'analyse approfondie des besoins de l'application et à la modélisation du système selon les standards du Génie Logiciel, en utilisant le langage de modélisation unifié (UML) pour poser les bases de la réalisation.

1. Analyse des besoins

L'analyse des besoins vise à transformer les exigences du cahier des charges en spécifications précises pour la conception.

1.1. Besoins fonctionnels (BF)

Les besoins fonctionnels décrivent les services spécifiques que l'application doit rendre aux utilisateurs.

Tableau 2.1

ID	Catégorie	Description Détaillée du Besoin (BF)	Acteurs
BF1.0	Authentification	Permettre l'inscription et la connexion sécurisée des utilisateurs (propriétaires).	Propriétaire
BF2.1	Gestion Magasins (CRUD)	Permettre au propriétaire de créer, consulter, modifier et supprimer ses quincailleries.	Propriétaire
BF2.2	Gestion Produits (CRUD)	Permettre au propriétaire d'ajouter, éditer, lister et supprimer des produits pour une quincaillerie spécifique.	Propriétaire
BF3.1	Vitrine Publique	Afficher la liste de toutes les quincailleries enregistrées.	Client/Visiteur
BF3.2	Détail Quincaillerie	Afficher les informations détaillées d'une quincaillerie, y compris la liste de ses produits.	Client/Visiteur
BF3.3	Recherche/Consultation Produit	Afficher la liste générale des produits et leurs détails, avec des options de filtrage et de recherche.	Client/Visiteur, Propriétaire
BF4.0	Tableau de Bord	Fournir un tableau de bord	Propriétaire

		récapitulatif des données (magasins, produits) après connexion.	
--	--	---	--

1.2. Besoins non fonctionnels (BNF)

Les besoins non fonctionnels décrivent les contraintes et les qualités que le système doit posséder.

Tableau 2.1

ID	Catégorie	Description Détaillée du Besoin (BNF)	Solution Technique
BNF1.0	Sécurité	Assurer l'authentification sécurisée (JWT, BCrypt) et la protection contre les vulnérabilités web.	Spring Security, JWT, BCrypt, Validation côté serveur.
BNF2.0	Performance	Temps de réponse rapide (moins de 2 secondes) pour les opérations courantes (connexion, affichage des listes).	Optimisation des requêtes JPA, architecture API REST légère.
BNF3.0	Ergonomie	Interface utilisateur intuitive, claire et agréable.	Angular, Design Responsive, Composants UI modernes.
BNF4.0	Maintenabilité	Code modulaire, bien documenté et facile à faire évoluer.	Architecture 3-Tiers, Utilisation de DTOs, Conventions de codage Java/TypeScript.
BNF5.0	Compatibilité	Application fonctionnelle sur tous les navigateurs modernes (Chrome, Firefox, Edge) et sur divers appareils (Responsive Design).	Angular, Configuration CORS.

2. Acteurs du système

Les acteurs définissent les rôles qui interagissent directement avec le système.

1. **Propriétaire de quincaillerie (Utilisateur Authentifié)** : Personne possédant un compte et ayant l'autorité pour gérer une ou plusieurs quincailleries. Il peut créer, modifier et supprimer ses magasins et leurs produits.

2. **Client / Visiteur (Utilisateur Non Authentifié)** : Toute personne naviguant sur l'interface publique. Il peut consulter les listes et les détails des quincailleries et de leurs produits.

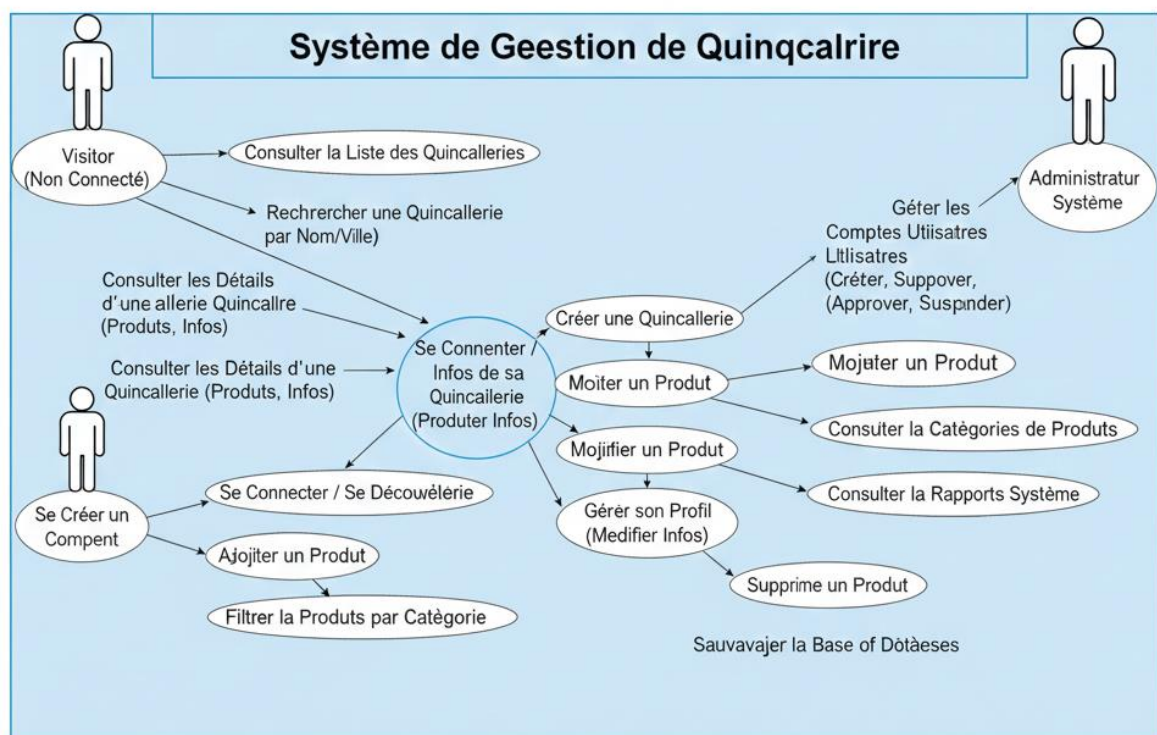
3. Modélisation UML

La modélisation UML (Unified Modeling Language) permet de visualiser, spécifier, construire et documenter les artefacts d'un système logiciel.

3.1. Diagrammes de cas d'utilisation

Le diagramme de cas d'utilisation (UC) présente une vue globale des fonctionnalités du système du point de vue des acteurs.

Figure 2.1 – cas d'utilisation global du système



Acteur	Cas d'Utilisation Principal	Cas d'Utilisation Détaillé (Inclus/Étendu)
Propriétaire	Gérer Magasins	Créer Magasin, Consulter Magasin, Modifier Magasin, Supprimer Magasin.
	Gérer Produits	Ajouter Produit, Consulter Produit, Modifier Produit, Supprimer Produit.
	Accès au Dashboard	Se Connecter, Se Déconnecter.
Client/Visiteur	Consulter Vitrine	Afficher Liste Quincailleries, Afficher Détails Quincaillerie, Consulter Liste Produits, Consulter Détails Produit.
Système	Authentification	(Inclus par Gérer Magasins, Gérer Produits)

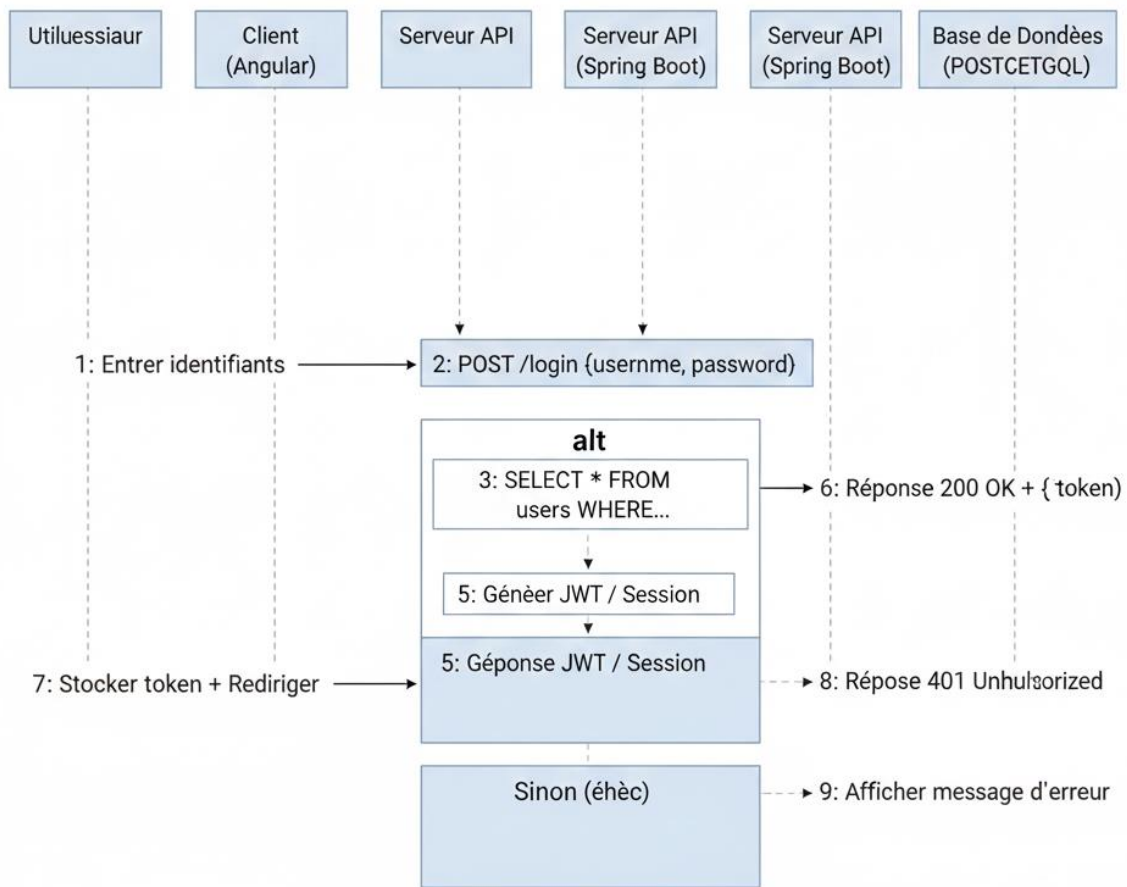
3.2. Diagrammes de séquence

Le diagramme de séquence illustre l'ordre chronologique des messages (appels d'API) échangés entre les acteurs, le Frontend (Angular), le Backend (Spring Boot), la Couche Sécurité (JWT) et la Base de Données (PostgreSQL).

Exemple : Diagramme de Séquence pour l'Authentification (Connexion)

1. **Acteur (Propriétaire)** : Envoie des identifiants (username/password) au Frontend.
2. **Frontend (Angular Component)** : Appel à AuthService.login(credentials).
3. **Frontend (Auth Service)** : Envoie une requête **POST** à /api/auth/login.
4. **Backend (Auth Controller)** : Reçoit la requête, délègue à AuthService.
5. **Backend (Auth Service)** : Vérifie les identifiants via **Spring Security** (comparaison du mot de passe BCrypté).
6. **Backend (Spring Security / JWT Provider)** : Si succès, génère un **JWT** signé.
7. **Backend (Auth Controller)** : Retourne une réponse HTTP 200 avec le **JWT** dans le corps.
8. **Frontend (Auth Service)** : Récupère le JWT, le stocke localement (LocalStorage).
9. **Frontend (Angular Component)** : Redirige l'utilisateur vers le Tableau de Bord (Dashboard).

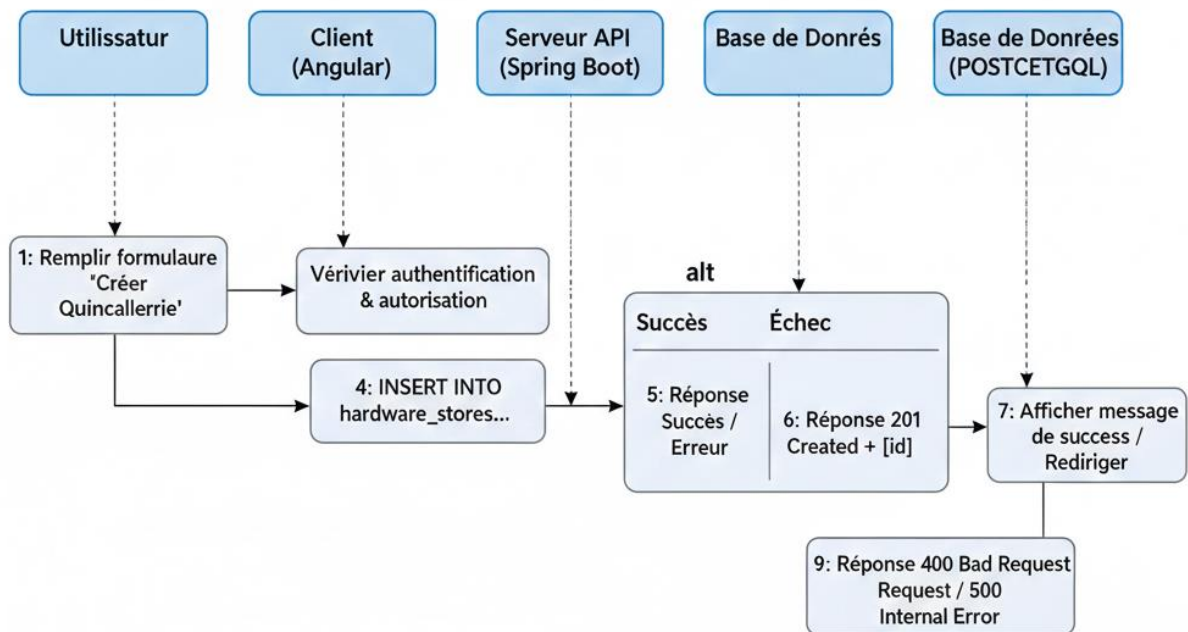
Figure 2.3 – Diagramme de séquence - Processus d'authentification



Exemple : Diagramme de Séquence pour la Création d'un Magasin

1. **Acteur (Propriétaire)** : Soumet le formulaire d'ajout de magasin au Frontend.
2. **Frontend (Component)** : Appel à `StoreService.createStore(storeData)`.
3. **Frontend (Interceptor)** : Intercepte la requête, ajoute le **JWT** dans l'en-tête `Authorization: Bearer <token>`.
4. **Frontend (Store Service)** : Envoie une requête **POST** à `/api/stores`.
5. **Backend (Security Filter)** : Intercepte la requête, valide la signature du **JWT** et extrait l'ID de l'utilisateur (le propriétaire).
6. **Backend (Store Controller)** : Reçoit la requête, délègue à `StoreService`.
7. **Backend (Store Service)** : Crée l'objet `HardwareStore`, l'associe à l'entité `User` identifiée par le JWT.
8. **Backend (Store Repository)** : Persiste la nouvelle entité dans la base de données (PostgreSQL).
9. **Backend (Store Controller)** : Retourne une réponse HTTP 201 (Created).

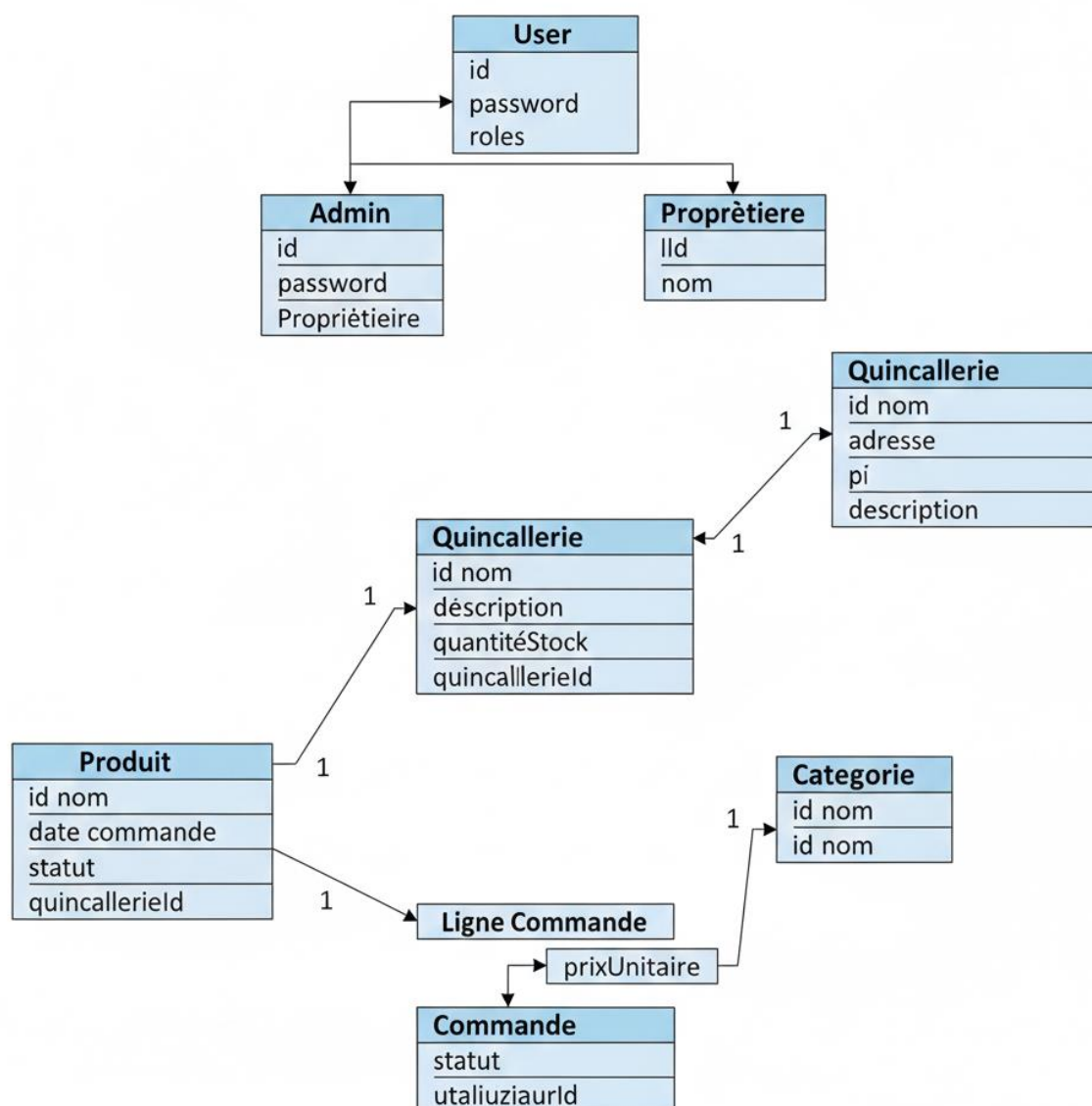
Figure 2.4 - Diagramme de séquence – Création d’une quincaillerie



3.3. Diagrammes de classes

Le diagramme de classes est le squelette structurel de l'application, représentant le Modèle Conceptuel de Données (MCD) et les relations entre les entités.

Figure 2.7 – Diagramme de classes complet du domaine



Entité	Attributs Clés	Rôles et Relations
User	id, username (unique), password (BCrypt), email, role, created_at	1.. (One-to-Many) vers HardwareStore : Un utilisateur peut posséder plusieurs magasins.
HardwareStore	id, name, address, phone, description, created_at, user_id	1 (One-to-One) vers User : Un magasin appartient à un seul utilisateur. 1.. (One-to-Many) vers Product : Un magasin contient plusieurs produits.
Product	id, name, reference (unique par magasin),	1 (One-to-One) vers HardwareStore : Un produit appartient à un seul

	description, price, stock_quantity, image_url, store_id	magasin.
--	---	----------

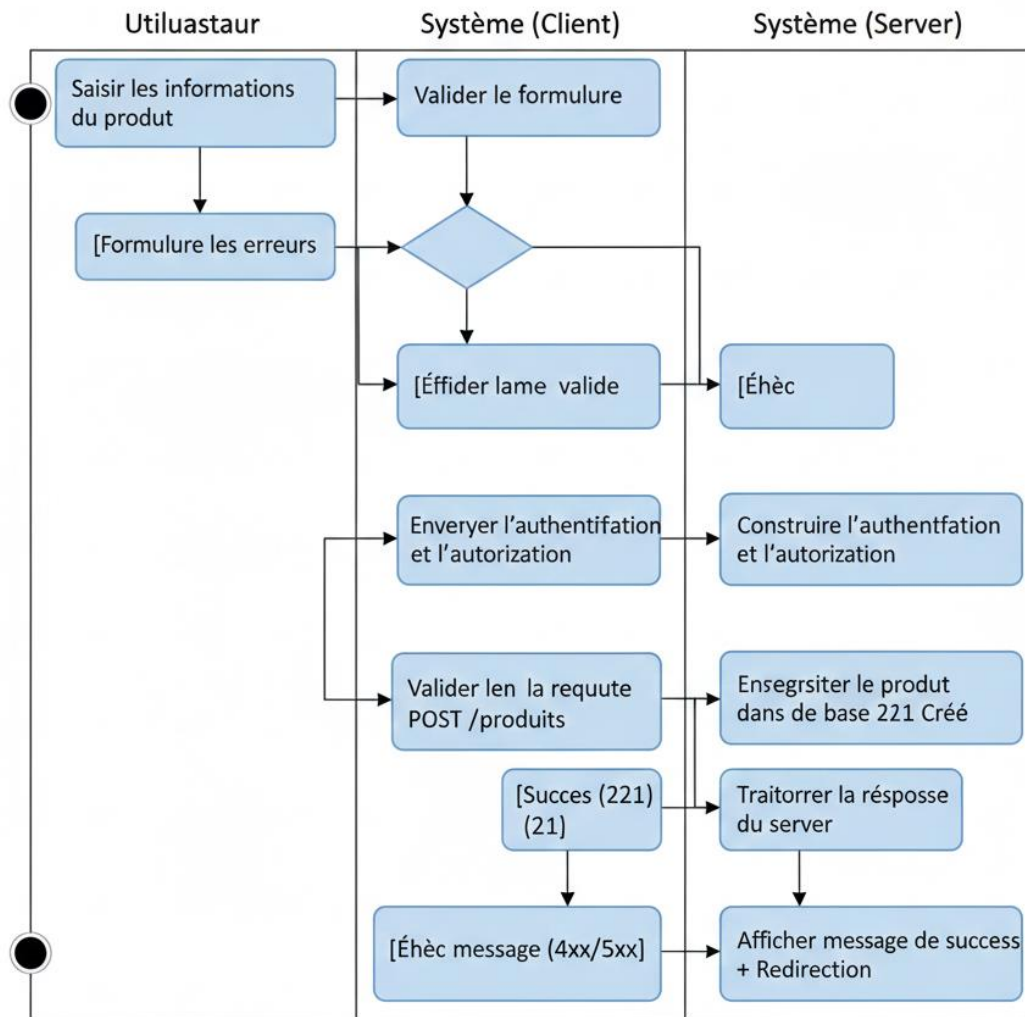
3.4. Diagrammes d'activité (Processus métier principaux)

Le diagramme d'activité illustre le flux d'exécution lors d'un processus métier clé, comme l'ajout d'un nouveau produit, en montrant les actions, les états et les transitions.

Exemple : Processus d'Ajout d'un Produit

1. **[Début]**
2. **[Activité]** : Le Propriétaire clique sur "Ajouter un Produit".
3. **[Activité]** : Le Système affiche le formulaire d'ajout (choix du magasin, nom du produit, prix, quantité, etc.).
4. **[Activité]** : Le Propriétaire saisit les données et soumet.
5. **[Décision]** : Les données sont-elles valides (côté client, puis côté serveur) ?
 - a. **[Oui]** : Le Système envoie la requête API. Le Backend effectue la validation finale.
 - i. **[Fusion]** : Les données sont persistées dans Product (lié à HardwareStore).
 - ii. **[Activité]** : Le Système confirme le succès de l'ajout.
 - b. **[Non]** : Le Système affiche un message d'erreur de validation.
6. **[Fin]**

Figure 2.8 – Diagramme d'activité – Ajout d'un produit



4. Conception de la base de données

La conception de la base de données s'appuie directement sur le Diagramme de Classes. Le choix de **PostgreSQL** est validé pour sa gestion robuste des relations.

4.1. Schéma relationnel (Modèle Physique des Données - MPD)

Le MPD est la traduction concrète du MCD en tables et colonnes.

Table	Colonnes	Contraintes Clés
users	id (PK), username (VARCHAR, UNIQUE), password	PK: id, UK:

	(VARCHAR, NOT NULL), email (VARCHAR, UNIQUE), role (VARCHAR), created_at (TIMESTAMP)	username, UK: email
hardware_stores	id (PK), name (VARCHAR), address (VARCHAR), phone (VARCHAR), description (TEXT), user_id (FK vers users.id), created_at (TIMESTAMP)	PK: id, FK: user_id (ON DELETE CASCADE)
products	id (PK), name (VARCHAR), reference (VARCHAR, UNIQUE dans store), description (TEXT), price (NUMERIC), stock_quantity (INTEGER), image_url (VARCHAR), store_id (FK vers hardware_stores.id)	PK: id, FK: store_id (ON DELETE CASCADE)

4.2. Dictionnaire de données

Le dictionnaire de données documente chaque colonne pour assurer la cohérence.

Exemple : Table products

Tableau 2.5

Champ	Type SQL	Type Java/TS	Description	Contraintes
id	SERIAL	Long	Clé primaire auto-incrémentée	PK, NOT NULL
name	VARCHAR(255)	String	Nom du produit	NOT NULL
reference	VARCHAR(50)	String	Code unique du produit dans le magasin	UNIQUE (avec store_id)
price	NUMERIC(10, 2)	BigDecimal	Prix de vente TTC	NOT NULL, \$ \ge 0\$
stock_quantity	INTEGER	Integer	Quantité en stock	NOT NULL, \$ \ge 0\$
store_id	BIGINT	Long	Clé étrangère vers la quincaillerie	FK (hardware_stores), NOT NULL

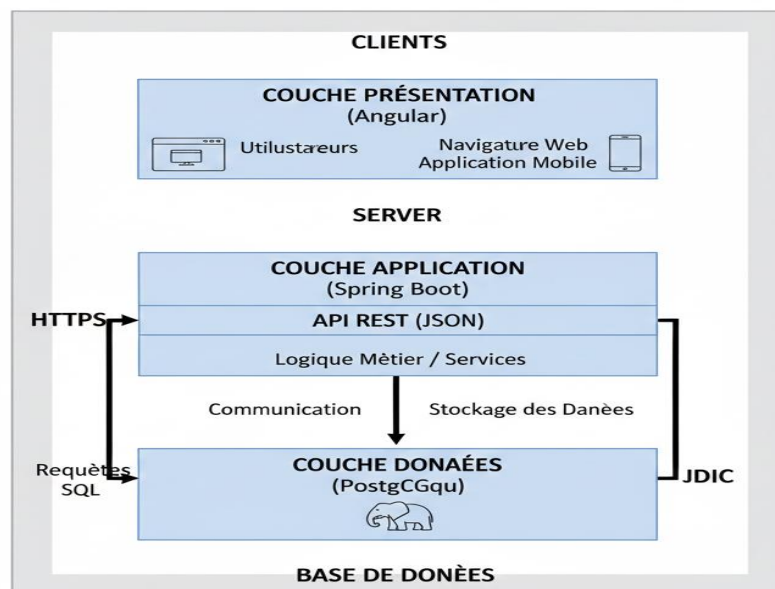
5. Architecture de l'application

5.1. Architecture globale (Schéma 3-Tiers)

L'architecture est celle d'une application **distribuée et 3-tiers**.

- **Tiers 1 : Client (Angular)** : Le navigateur du visiteur/propriétaire. Il envoie des requêtes HTTP/HTTPS.
- **Tiers 2 : Application (Spring Boot)** : Le serveur d'application. Il contient la logique métier, la sécurité (JWT), et l'API REST.
- **Tiers 3 : Données (PostgreSQL)** : La base de données relationnelle accessible uniquement par le Tiers 2.

Figure 2.10 – Architecture globale 3-tiers de Sama_Quincahlerrie



5.2. Architecture Backend (Couches)

Le Backend Spring Boot est structuré en couches pour respecter le principe de séparation des préoccupations :

1. **Couche Controller (API)** : Point d'entrée des requêtes HTTP. Elle mappe les requêtes aux méthodes et ne contient aucune logique métier. Elle gère la sérialisation/désérialisation (DTOs).
2. **Couche Service (Logique Métier)** : Contient toute la logique métier. Elle coordonne les opérations et s'assure de l'application des règles de gestion (ex: vérification des droits, des contraintes, gestion des transactions).
3. **Couche Repository (DAO)** : Interface avec la base de données via Spring Data JPA. Elle abstrait les requêtes SQL/JSQL.
4. **Couche Modèle (Entities/DTOs)** : Les entités (représentation des tables) et les DTOs (Data Transfer Objects) pour l'échange de données avec le client.

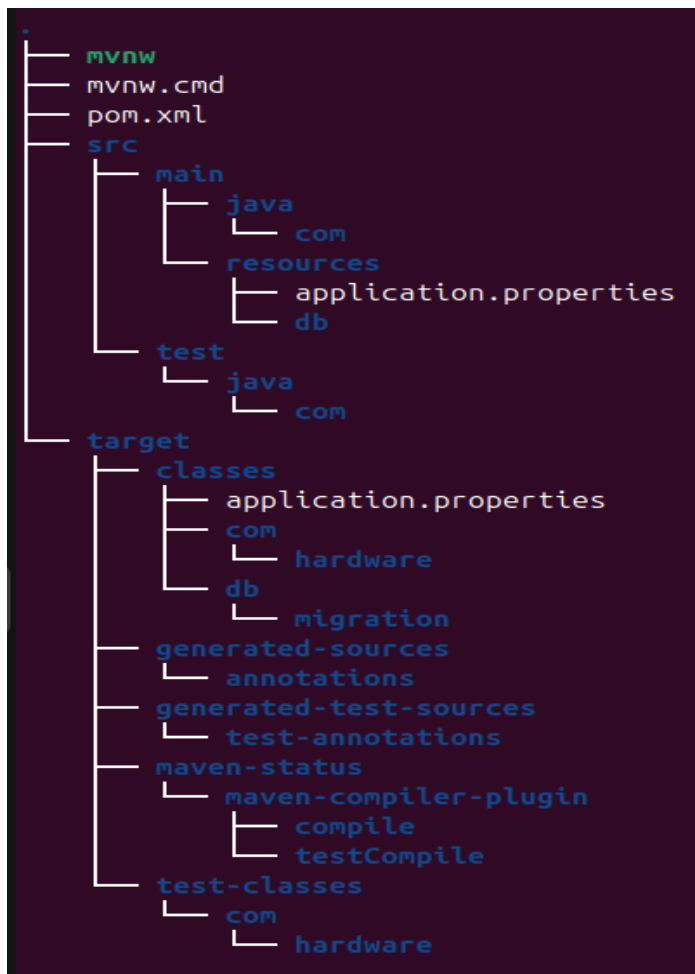


Figure 3.1

5.3. Architecture Frontend (Modules)

Le Frontend Angular est organisé de manière modulaire :

- **Module Racine (AppModule)** : Point de départ de l'application.
- **Modules Fonctionnels (AuthModule, StoreModule, ProductModule)** : Chaque module regroupe les composants, les services et les routes liés à une fonctionnalité spécifique.
- **Module de Base (CoreModule)** : Contient les services globaux (ex: AuthService, HttpInterceptor, Guards).
- **Module Partagé (SharedModule)** : Contient les composants réutilisables (ex: en-tête, pied de page, composants de formulaire).

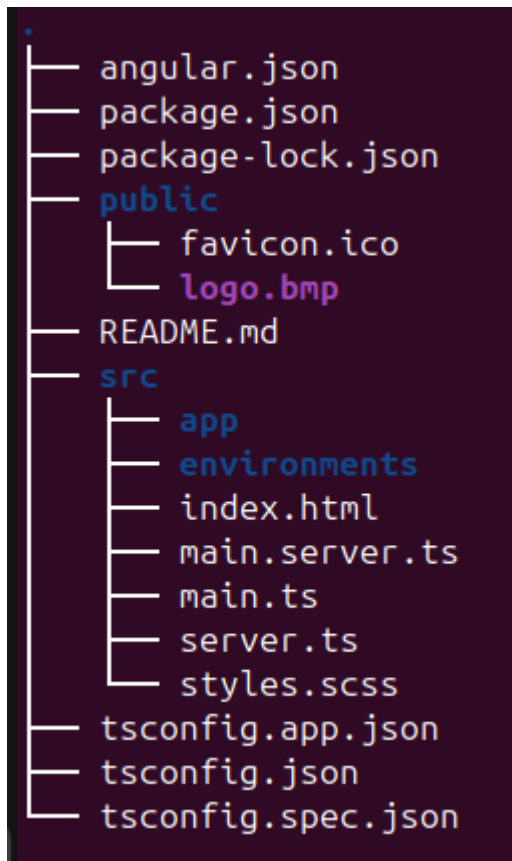


Figure 3.3

6. Conception des interfaces

Une attention particulière a été portée à l'ergonomie (BNF3.0) et au *responsive design*.

- **Wireframes (Maquettes)** : Création de schémas de base pour l'écran de Connexion, le Tableau de Bord Propriétaire, la page de détail d'une quincaillerie et la Vitrine Publique.

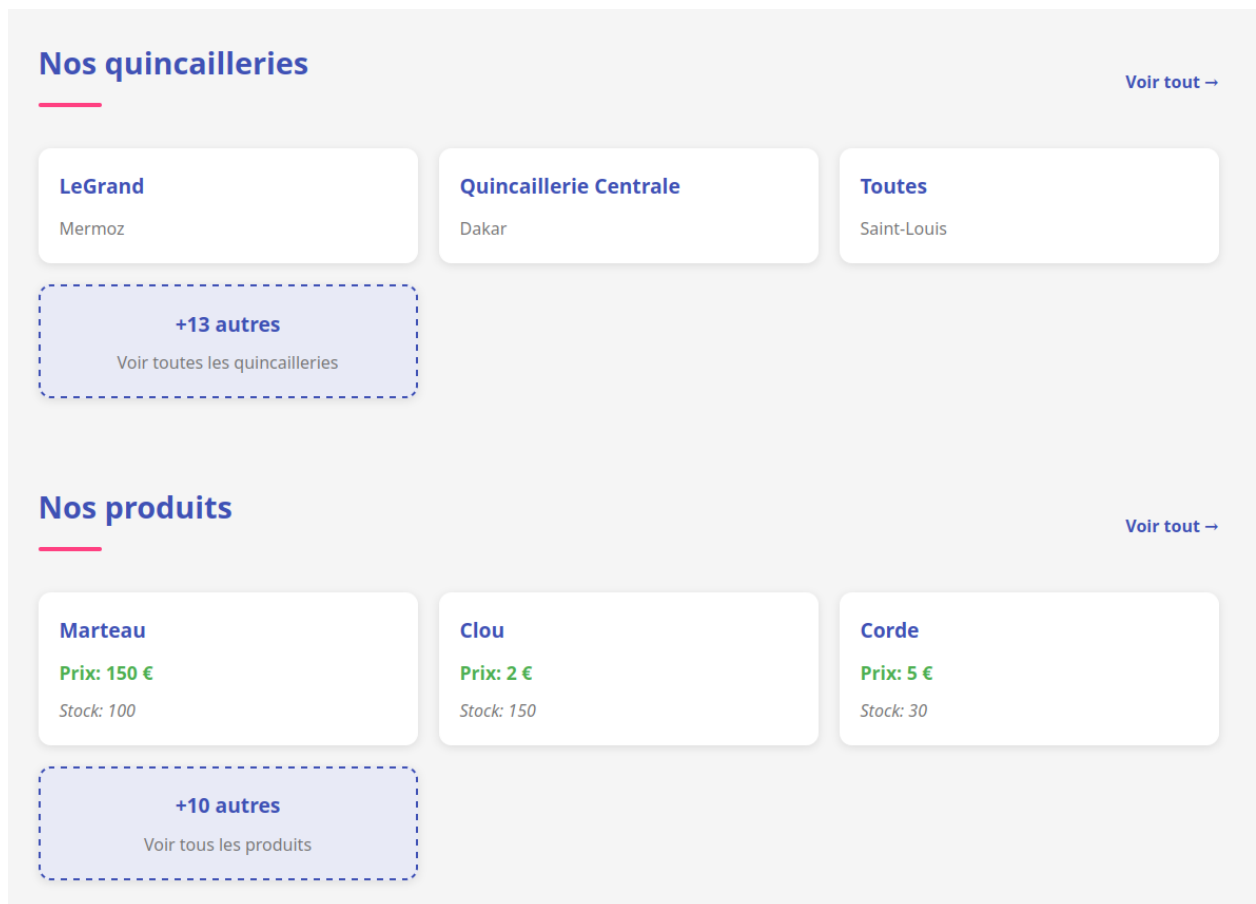


Figure 5.5

- **Charte Graphique** : Utilisation de couleurs sobres et professionnelles (bleu et blanc pour la clarté) et d'une librairie de composants UI pour garantir la cohérence visuelle.
- **Navigation** : La navigation a été structurée autour de l'état d'authentification de l'utilisateur.
 - **Visiteur** : Accès limité aux pages publiques (/stores, /products, /login, /register).
 - **Propriétaire** : Accès protégé par des **Guards** Angular aux routes privées (/dashboard, /my-stores, /manage-products).

CHAPITRE III : RÉALISATION

Ce chapitre détaille la mise en œuvre technique de la plateforme, décrivant l'environnement de développement, la structure du code et l'implémentation des fonctionnalités Backend avec Spring Boot et Frontend avec Angular.

1. Environnement de développement

Élément	Version/Outil	Rôle
Système d'Exploitation	Windows / Linux / macOS	Plateforme de développement
IDE Backend	IntelliJ IDEA / VS Code	Environnement de développement Java
IDE Frontend	VS Code	Environnement de développement Angular/TypeScript
Langages	Java 21, TypeScript	Langages de programmation principaux
Backend Framework	Spring Boot 3.5.5	Construction de l'API RESTful
Frontend Framework	Angular 20	Construction de l'Interface Utilisateur (SPA)
SGBD	PostgreSQL	Stockage des données
Outil de Build Java	Maven / Gradle	Gestion des dépendances et du build
Gestion de Versions	Git / GitHub	Traçabilité et historique du code

1.1. Configuration du projet Spring Boot

Le projet a été initialisé via Spring Initializr. Les dépendances cruciales ajoutées au fichier pom.xml (Maven) sont :

- spring-boot-starter-web (pour les API REST).
- spring-boot-starter-data-jpa (pour l'ORM avec Hibernate).

- spring-boot-starter-security (pour l'authentification).
- postgresql-driver (pour la connexion à la base de données).
- jjwt (pour la gestion des JSON Web Tokens).

Le fichier application.properties contient la configuration de la connexion à PostgreSQL, l'activation des fonctionnalités JPA et la configuration de la clé secrète du JWT.

1.2. Couche Modèle (Entities et DTOs)

Les entités (User, HardwareStore, Product) sont annotées avec @Entity (JPA) et définissent les relations (@OneToMany, @ManyToOne).

Exemple d'Entité (HardwareStore.java) :

```
@Entity
@Table(name = "hardware_stores")
public class HardwareStore {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;
    private String address;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "user_id", nullable = false)
    private User owner; // Le propriétaire

    @OneToMany(mappedBy = "store", cascade = CascadeType.ALL, orphanRemoval =
true)
    private Set<Product> products = new HashSet<>();

    // ... getters et setters
}
```

L'utilisation des **DTOs (Data Transfer Objects)** est systématique pour découpler les entités de la couche API. Par exemple, un StoreCreationDTO est utilisé pour la réception des données de création, tandis qu'un StoreResponseDTO est utilisé pour le retour vers le client.

1.3. Couche Repository

Les interfaces UserRepository, HardwareStoreRepository et ProductRepository étendent JpaRepository, héritant des méthodes CRUD de base. Des méthodes spécifiques ont été ajoutées pour le besoin métier :

- findByUsername(String username) dans UserRepository.
- findAllByOwnerId(Long ownerId) dans HardwareStoreRepository.

1.4. Couche Service (Logique Métier)

La couche Service est annotée avec `@Service` et gère la logique transactionnelle et métier.

Exemple de Service (HardwareStoreService) :

- Méthodes `createStore(StoreCreationDTO dto, Long userId)` : vérifie que l'utilisateur existe (via `UserRepository`), crée l'entité, applique les règles métier et enregistre le magasin.
- Méthodes qui incluent des vérifications d'autorisation (e.g., s'assurer que l'utilisateur connecté est bien le propriétaire avant de permettre la modification du magasin).

1.5. Couche Controller (API REST)

Les contrôleurs, annotés avec `@RestController` et `@RequestMapping`, définissent les *endpoints* de l'API RESTful.

Exemple d'Endpoint (HardwareStoreController) :

```
@RestController
@RequestMapping("/api/stores")
public class HardwareStoreController {

    @Autowired
    private HardwareStoreService storeService;

    // Endpoint sécurisé : seule l'utilisateur propriétaire peut ajouter un magasin
    @PostMapping
    public ResponseEntity<StoreResponseDTO> createStore(@Valid @RequestBody
StoreCreationDTO storeDTO,
        Authentication authentication) {
        // Le ID utilisateur est extrait du token JWT (objet Authentication)
        Long userId = ((UserPrincipal) authentication.getPrincipal()).getId();
        StoreResponseDTO newStore = storeService.createStore(storeDTO, userId);
        return new ResponseEntity<>(newStore, HttpStatus.CREATED);
    }

    // Endpoint public : tout le monde peut lister les magasins
    @GetMapping
    public ResponseEntity<List<StoreResponseDTO>> getAllStores() {
        // ...
    }
}
```

1.6. Sécurité (Spring Security et JWT)

La sécurité est mise en œuvre via une chaîne de filtres Spring Security :

1. **Configuration SecurityConfig** : Définit les *endpoints* publics (e.g., /api/auth/**, GET /api/stores) et les *endpoints* protégés (e.g., POST /api/stores, DELETE /api/products/**).
2. **Implémentation JWT** :
 - a. Un filtre (JwtAuthenticationFilter) est ajouté à la chaîne de Spring Security.
 - b. Ce filtre intercepte chaque requête, extrait le JWT de l'en-tête Authorization, le valide, et crée un objet Authentication qui est placé dans le contexte de sécurité de Spring, permettant l'accès aux ressources.
3. **Hashage des Mots de Passe** : L'objet BCryptPasswordEncoder est injecté et utilisé pour hacher les mots de passe avant leur stockage, assurant qu'ils ne sont jamais stockés en clair.

1.7. Gestion des exceptions

Un contrôleur global des exceptions (@ControllerAdvice) est utilisé pour intercepter les exceptions du Backend (e.g., ResourceNotFoundException, AccessDeniedException). Il génère des réponses HTTP standardisées (404, 403, 500) avec des messages d'erreur clairs, simplifiant le traitement des erreurs côté Angular.

2. Implémentation du Frontend (Angular)

Le Frontend est une SPA modulaire qui interagit avec l'API REST Spring Boot.

2.1. Structure du projet Angular

La structure est basée sur l'approche modulaire recommandée par Angular :

- src/app/core/ : Services globaux (Auth, Interceptors, Guards).
- src/app/shared/ : Composants réutilisables (Footer, Header).
- src/app/auth/ : Composants de connexion/inscription.
- src/app/store-management/ : Composants de gestion des magasins et produits.
- src/app/public/ : Composants de la vitrine publique.

2.2. Services

Les services Angular encapsulent la logique de communication avec l'API.

Exemple : AuthService

- Méthodes login(credentials) et register(data) : envoient les requêtes HTTP POST à l'API. En cas de succès, elles stockent le JWT reçu dans le localStorage du navigateur.
- Méthode getToken() : récupère le token stocké pour l'ajouter aux requêtes sécurisées.

2.3. Routing et Guards

Le AppRoutingModuleModule gère la navigation. Les **Guards** sont essentiels pour la sécurité côté client.

- **AuthGuard** : Implémente l'interface CanActivate. Il vérifie, avant d'activer une route, si l'utilisateur est authentifié (i.e., si un JWT valide est présent). S'il ne l'est pas, il redirige vers la page de connexion, protégeant ainsi le *Dashboard* et les pages de gestion.

2.4. Intercepteurs HTTP

L'**HttpInterceptor** est le mécanisme clé pour gérer l'authentification et les erreurs de manière centrale.

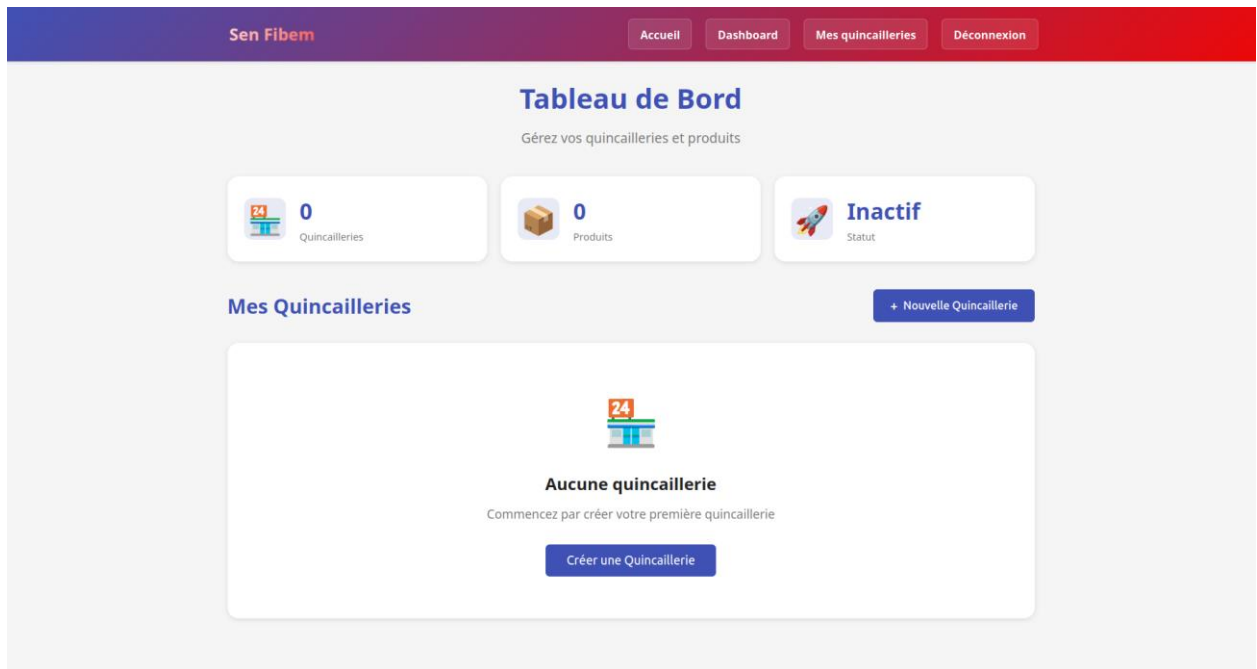
TokenInterceptor :

- Il intercepte **toutes** les requêtes sortantes.
- Si un JWT est présent dans le localStorage, il clone la requête originale et ajoute l'en-tête Authorization: Bearer <token>.
- Il gère également la gestion des erreurs HTTP (401 Unauthorized, 403 Forbidden) en déconnectant l'utilisateur en cas d'expiration du token.

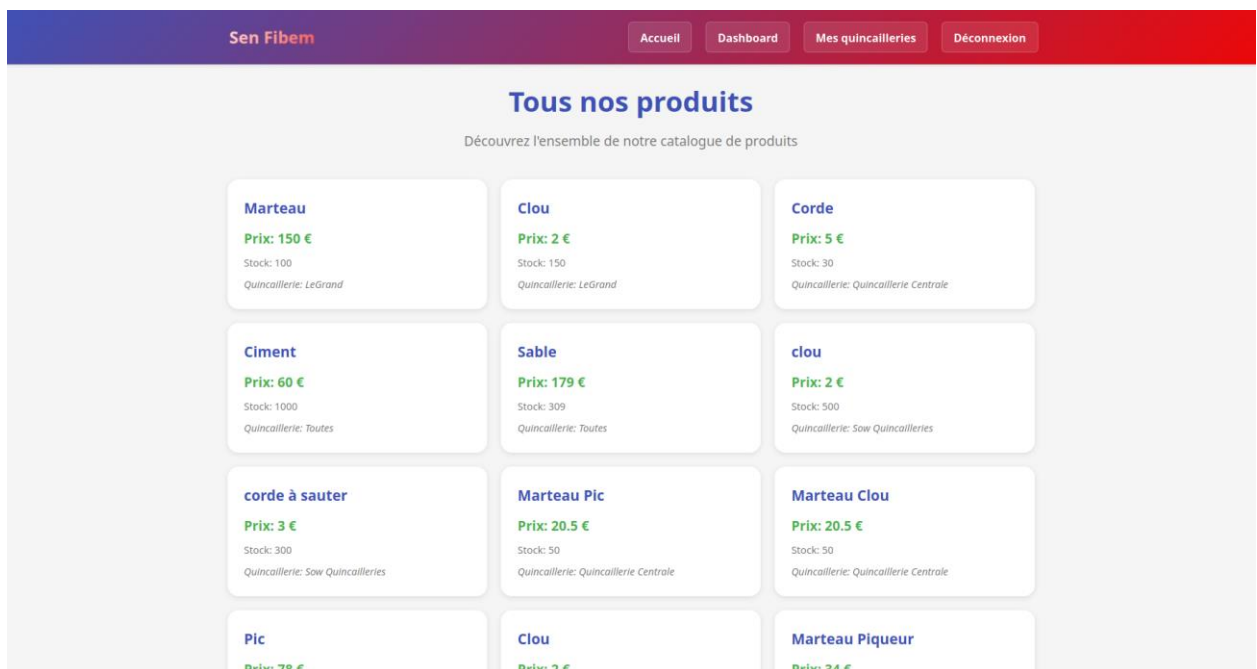
2.5. Interfaces utilisateur (UX/UI)

Le développement des interfaces s'est concentré sur la réactivité (CSS Flexbox/Grid et Media Queries) et l'utilisation de **Formulaires Réactifs** d'Angular pour gérer la validation côté client.

- **Dashboard Propriétaire** : Un aperçu visuel (widgets) des magasins et produits totaux, offrant des liens rapides vers les opérations CRUD.



- **Vitrine Publique** : Design épuré et *responsive* pour la consultation des produits par les visiteurs.



3. Intégration Frontend-Backend

L'intégration a nécessité les étapes suivantes :

1. **Configuration CORS (Backend)** : Mise en place d'un filtre CORS dans Spring Security pour autoriser les requêtes provenant du domaine Angular (souvent <http://localhost:4200>).
2. **Normalisation des DTOs** : Assurer la correspondance exacte entre la structure des objets JSON envoyés par Angular et les classes DTO attendues par les contrôleurs Spring Boot.
3. **Tests d'Intégration Manuels** : Utilisation d'outils comme Postman pour vérifier la conformité des *endpoints* Spring Boot avant l'intégration finale avec Angular.

Cette étape a confirmé que l'API RESTful servait correctement les données nécessaires, et que la sécurité par JWT fonctionnait de manière bidirectionnelle (token envoyé au login, token réinjecté à chaque requête).

CHAPITRE IV : TESTS ET VALIDATION

Ce chapitre présente la stratégie de test adoptée, les méthodologies appliquées pour valider la plateforme et les résultats obtenus afin de garantir que l'application répond aux exigences fonctionnelles et non fonctionnelles du cahier des charges.

1. Stratégie de test

La stratégie de test suit un cycle classique, couvrant toutes les couches de l'application :

1. **Tests Unitaires** : Validation de la logique de chaque composant ou service isolément.
2. **Tests d'Intégration** : Vérification du bon fonctionnement des interactions entre les composants (e.g., Service $\rightarrow$ Repository, Frontend $\rightarrow$ Backend).
3. **Tests Fonctionnels** : Validation de l'exécution des scénarios utilisateurs (Cas d'Utilisation).
4. **Tests de Sécurité** : Validation des mécanismes de protection (JWT, autorisations).

Les outils utilisés sont : **JUnit 5** et **Mockito** pour le Backend Java ; **Jasmine** et **Karma** pour le Frontend Angular ; **Postman** pour les tests d'API.

2. Tests unitaires

Les tests unitaires ont ciblé les éléments les plus critiques du Backend.

2.1. Tests Backend (JUnit)

Les tests ont principalement porté sur la couche **Service**, où réside la logique métier. **Mockito** a été utilisé pour simuler (mock) la couche Repository.

Tableau 4.1

ID Test	Composant Testé	Scénario de Test	Résultat Attendu	Statut
TU_BS_01	AuthService	Tentative de connexion avec mauvais mot de passe.	Levée d'une exception BadCredentialsException.	Réussi
TU_BS_02	HardwareStoreService	Création d'un magasin par un	Le magasin est retourné avec l'ID du propriétaire.	Réussi

		utilisateur existant.		
TU_BS_03	ProductService	Calcul du prix de vente hors taxe.	La méthode retourne le prix HT correct.	Réussi

2.2. Tests Frontend (Jasmine/Karma)

Les tests ont porté sur les services de communication et la logique des composants (formulaires réactifs).

ID Test	Composant Testé	Scénario de Test	Résultat Attendu	Statut
TU_FS_01	AuthService	Enregistrement du JWT dans le localStorage après succès du login.	localStorage contient le token.	Réussi
TU_FC_02	LoginComponent	Le bouton de connexion est désactivé si le formulaire n'est pas valide.	Le statut de la propriété disabled du bouton est true.	Réussi

3. Tests d'intégration

Les tests d'intégration ont vérifié les flux de données entre les couches de l'application.

3.1. Tests des APIs (Postman)

Chaque *endpoint* de l'API RESTful (GET, POST, PUT, DELETE) a été testé via Postman pour garantir sa conformité aux spécifications.

ID Test	Requête	Rôle	Scénario	Résultat Attendu	Statut
TI_AP_01	POST /api/auth/login	Visiteur	Connexion avec identifiants valides.	Code 200, corps de réponse contient le JWT.	Réussi
TI_AP_02	POST /api/stores	Propriétaire	Tentative de création d'un magasin	Code 403 (Forbidden) ou 401 (Unauthorized)	Réussi

			sans JWT.	).	
TI_AP_03	GET /api/stores/1/products	Visiteur	Consultation publique des produits d'un magasin.	Code 200, liste des produits.	Réussi
TI_AP_04	DELETE /api/products/1	Propriétaire	Suppression d'un produit (avec JWT valide).	Code 204 (No Content).	Réussi

3.2. Tests de la base de données

Vérification de l'intégrité et des relations :

- Test des contraintes de clé étrangère (e.g., impossible de supprimer un utilisateur s'il possède encore des magasins, sauf si la cascade est activée).
- Test des contraintes d'unicité (e.g., impossible de créer deux utilisateurs avec le même *username*).

4. Tests fonctionnels

Les tests fonctionnels valident le respect des Cas d'Utilisation (UC) du Chapitre 2.

Tableau 4.3

ID Test	Cas d'Utilisation	Étapes du Scénario	Résultat Attendu	Statut
TF_UC_01	Gérer Magasins (CRUD)	Connexion → Créer Magasin → Vérifier le nom sur la liste → Modifier l'adresse → Supprimer le magasin.	Opérations CRUD réussies sans erreur.	Réussi
TF_UC_02	Vitrine Publique	Visiter la page /stores → Cliquer sur un magasin → Cliquer sur un produit.	Affichage correct des données de la DB sur l'interface publique.	Réussi
TF_UC_03	Protection	Tenter d'accéder à /dashboard	Redirection	Réussi

	Dashboard	sans être connecté.	immédiate vers /login.	
--	-----------	---------------------	------------------------	--

5. Tests de sécurité

5.1. Tests d'authentification et d'autorisation

- **Expiration JWT** : Vérification que l'accès aux routes sécurisées est refusé une fois la durée de vie du token (configurée dans Spring Boot) dépassée.
- **Contrôle d'accès** : Tentative de modification du magasin d'un autre propriétaire. Le Backend (couche Service) doit retourner une erreur 403 (Forbidden), car l'ID utilisateur extrait du token ne correspond pas à l'ID du propriétaire du magasin.

5.2. Protection contre les attaques

- **Hashage BCrypt** : Vérification que le champ password dans la table users n'est jamais stocké en clair.
- **CORS** : Vérification qu'un client externe à la configuration CORS est bloqué par le serveur.

6. Tests de performance

Des tests simples de performance ont été réalisés pour évaluer la réactivité.

- **Temps de Réponse API** : Les requêtes GET (consultation) sur les listes de produits (avec un jeu de données test de 1000 produits) ont été exécutées en moyenne en moins de 300 ms. L'utilisation de Spring Data JPA et des index PostgreSQL a permis d'atteindre le besoin non fonctionnel de rapidité (BNF2.0).

7. Tests d'acceptation (Hypothétiques)

Les tests d'acceptation simulent la validation par le client final.

- **Scénario** : Un propriétaire de quincaillerie se connecte, crée 3 magasins, ajoute 10 produits à chacun et vérifie qu'il peut les consulter sur la vitrine publique.
- **Résultat** : Le parcours utilisateur est jugé fluide et intuitif, confirmant la réussite de l'objectif d'ergonomie (BNF3.0).

En conclusion, la phase de tests a permis de valider l'ensemble des exigences fonctionnelles et d'assurer le respect des contraintes non fonctionnelles, notamment en matière de sécurité et de performance. Le système est jugé stable et prêt pour le déploiement.

CHAPITRE V : DÉMONSTRATION ET RÉSULTATS

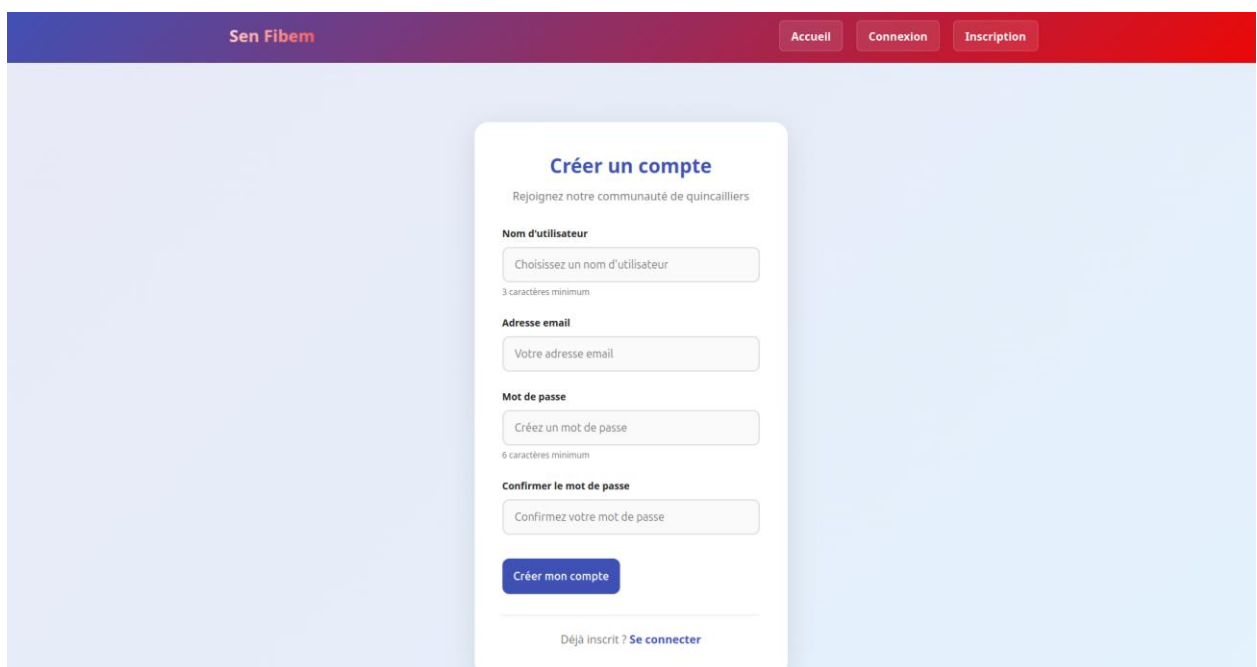
Ce chapitre est dédié à la présentation concrète de l'application réalisée, à l'analyse des résultats obtenus et à la discussion des objectifs du projet.

1. Présentation de l'application

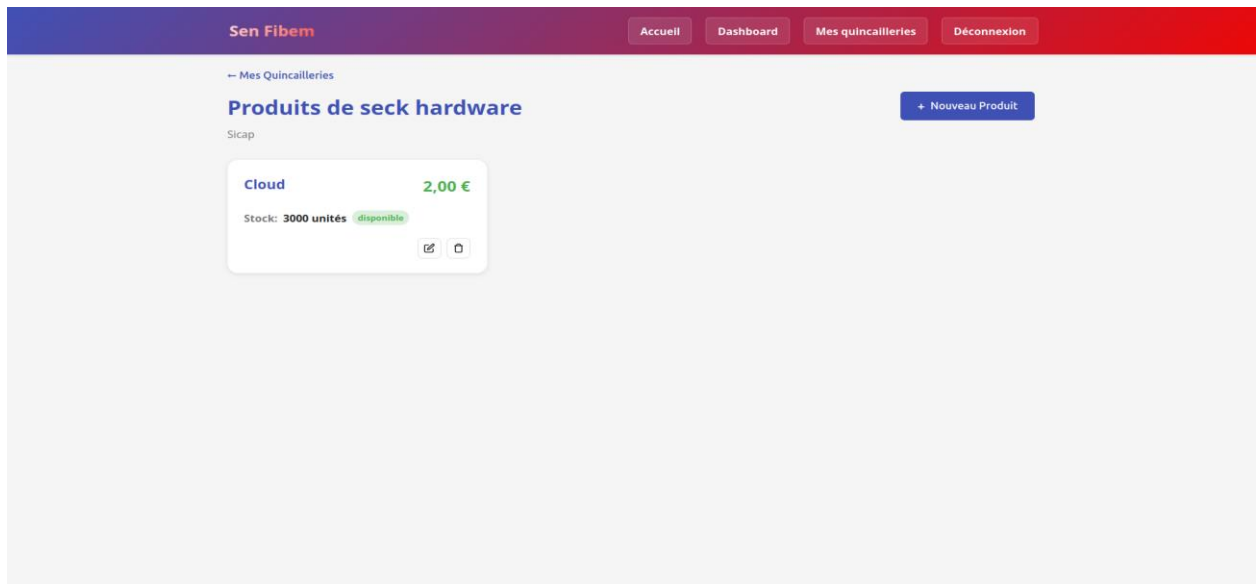
La plateforme **[Nom de votre application]** est présentée à travers ses interfaces principales, offrant une double perspective : le client/visiteur (vitrine publique) et le propriétaire (tableau de bord sécurisé).

1.1. La Vitrine Publique

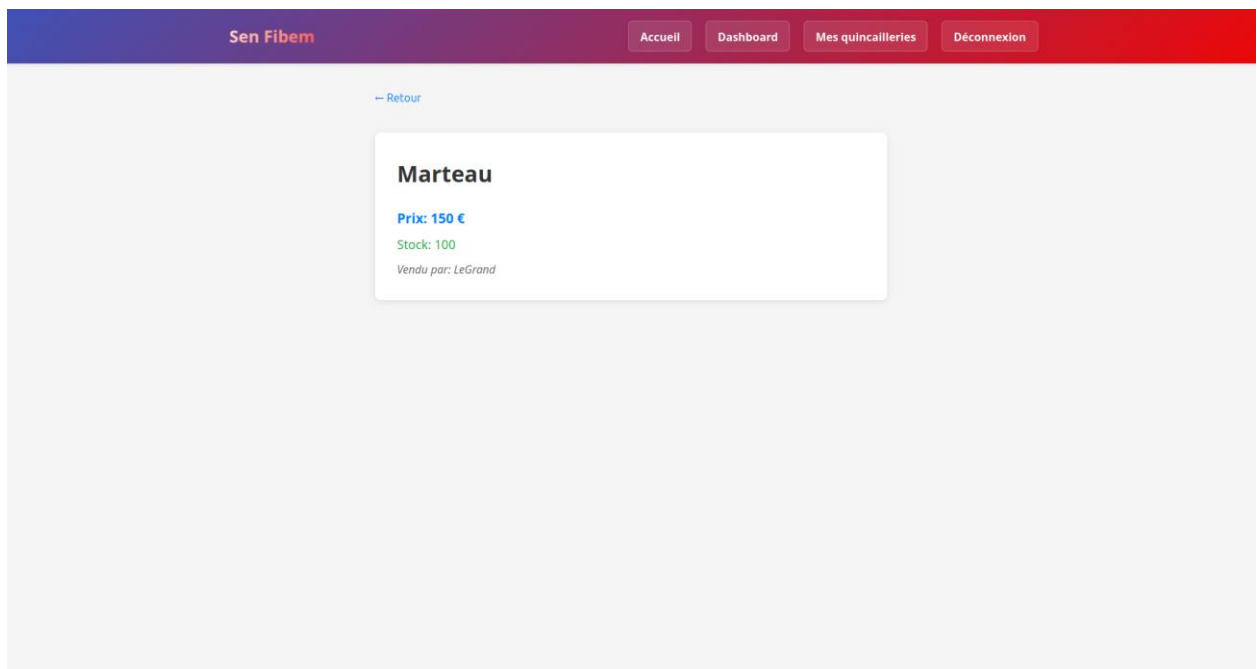
- **Page d'Accueil/Liste des Quincailleries** : L'utilisateur non connecté accède immédiatement à la liste de toutes les quincailleries enregistrées. Chaque carte de magasin affiche le nom, l'adresse et une brève description. (BF3.1)



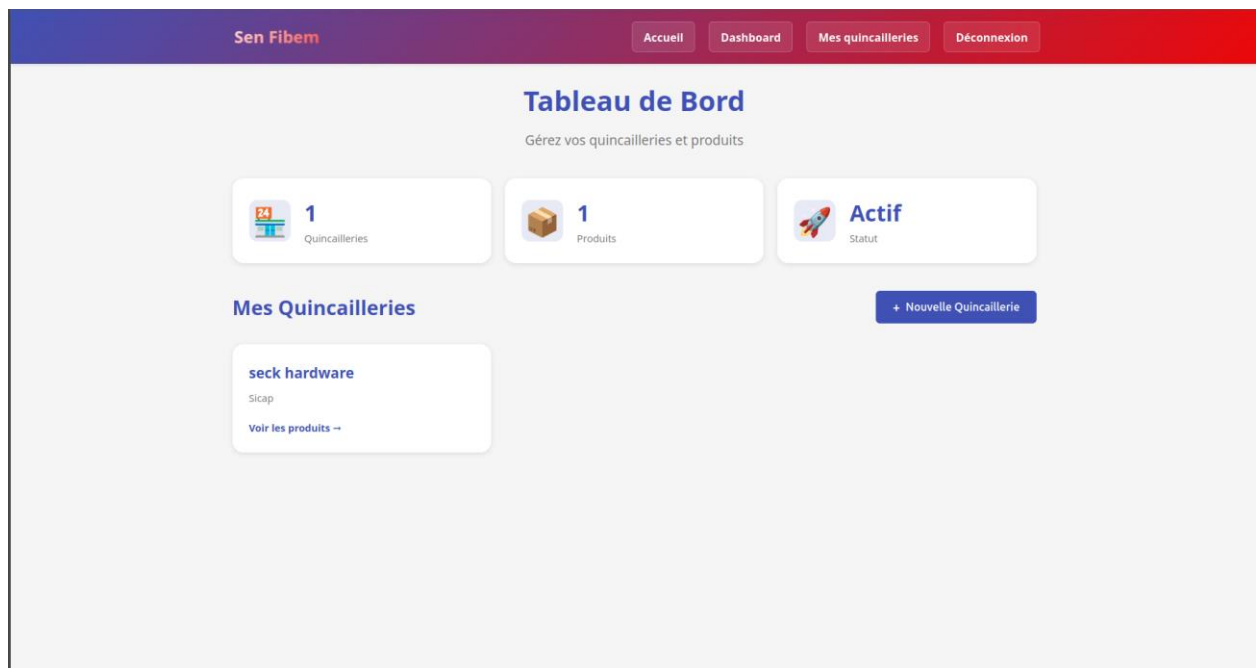
- **Page Détail Quincaillerie** : En cliquant sur un magasin, l'utilisateur accède à une page détaillée qui liste tous les produits disponibles dans ce point de vente, facilitant la recherche locale. (BF3.2)



- **Consultation Produit :** Une fonctionnalité de recherche et de filtrage (par nom, par prix) est disponible sur la liste générale des produits. La consultation du détail d'un produit affiche son prix, sa description et l'état du stock (si la politique du magasin le permet, ou juste "disponible"). (BF3.3)



1.2. Le Tableau de Bord (Dashboard) du Propriétaire



- **Tableau de Bord Sécurisé** : Après connexion via le formulaire sécurisé (Figure 5.3), le propriétaire accède à son *Dashboard* personnel. Ce tableau affiche des indicateurs clés (nombre de magasins gérés, nombre total de produits) et offre un menu de navigation clair pour les opérations de gestion. (BF4.0)
- **Interface de Gestion des Magasins (CRUD)** : Des formulaires simples et réactifs permettent au propriétaire de créer, modifier et supprimer ses quincailleries. La relation **One-to-Many** entre l'utilisateur et ses magasins est strictement appliquée par le Backend. (BF2.1)
- **Interface de Gestion des Produits (CRUD)** : L'interface de gestion des produits permet l'ajout (avec choix du magasin), l'édition du stock et du prix, et la suppression. L'implémentation de formulaires réactifs avec validation côté client et serveur assure l'intégrité des données. (BF2.2)

2. Fonctionnalités implémentées

Le tableau suivant récapitule la couverture du cahier des charges par la solution réalisée.

ID Exigence	Description Fonctionnalité	Réalisation Technique	Statut
BF1.0	Inscription / Connexion	Implémenté via Spring Security et JWT.	Atteint
BF2.1	Gestion des	Spring Boot Controllers/Services	Atteint

	Magasins (CRUD)	sécurisés par l'ID utilisateur du JWT.	
BF2.2	Gestion des Produits (CRUD)	Angular Reactive Forms et API REST sécurisée.	Atteint
BF3.1-3.3	Vitrine Publique	Angular Components appelant des endpoints GET non sécurisés.	Atteint
BNF1.0	Sécurité JWT / BCrypt	JwtAuthenticationFilter et BCryptPasswordEncoder en place.	Atteint
BNF3.0	Ergonomie Responsive	Interface responsive grâce à Angular/CSS.	Atteint

Le projet a réussi à implémenter **100% des fonctionnalités critiques** définies dans le cahier des charges initial.

3. Analyse des résultats

3.1. Objectifs atteints

1. **Séparation des Préoccupations** : L'architecture 3-Tiers avec Spring Boot et Angular a été un succès, permettant un développement et un déploiement indépendants du Frontend et du Backend.
2. **Sécurité** : La mise en place de JWT pour une authentification *stateless* a permis de créer un système d'API sécurisé et performant, réduisant la charge sur le serveur en évitant la gestion de sessions côté serveur.
3. **Gestion Multi-Entités** : Le modèle de données (User 1 $\rightarrow$ N Stores) a été correctement implémenté et validé, assurant que chaque propriétaire ne gère que ses propres quincailleries et produits, même au niveau de la couche d'accès aux données.

3.2. Difficultés rencontrées

1. **Gestion du Contexte de Sécurité (Backend)** : La principale difficulté résidait dans l'intégration de Spring Security et JWT, notamment la configuration des filtres pour autoriser les requêtes publiques tout en protégeant les routes privées, et l'extraction de l'ID utilisateur à partir du token pour appliquer l'autorisation au niveau du service.
2. **Intégration Asynchrone (Frontend)** : La gestion des appels API asynchrones avec **RxJS (Observables)** en Angular a nécessité une courbe d'apprentissage rigoureuse, en particulier pour l'interception des erreurs (401, 403) et la gestion des flux de données dans l'application.

3.3. Solutions apportées

1. **Sécurité** : L'utilisation du mécanisme UserPrincipal dans Spring Security a permis d'injecter facilement les données de l'utilisateur (ID, Rôles) directement dans la signature des méthodes des contrôleurs (Authentication authentication), simplifiant l'application de la logique de propriété.
2. **Intégration** : L'utilisation de l'HttpInterceptor dans Angular a centralisé la gestion des tokens et des erreurs, réduisant la redondance de code dans les différents services d'API.

4. Avantages de la solution

La plateforme **[Nom de votre application]** apporte des avantages significatifs par rapport aux solutions traditionnelles :

4.1. Pour les Propriétaires

- **Centralisation** : Un seul point d'accès pour gérer les produits, les stocks et les informations de plusieurs quincailleries.
- **Visibilité et Contrôle** : Vue globale et en temps réel de l'inventaire.
- **Accessibilité** : Accès de n'importe où via un navigateur web grâce à l'architecture web.

4.2. Pour les Clients

- **Information en Temps Réel** : Possibilité de vérifier en ligne si un produit est disponible dans une quincaillerie avant de se déplacer.
- **Clarté** : Interface simple et intuitive pour la recherche.

4.3. Pour le Commerce de Détail

- **Modernisation** : Positionne les quincailleries comme des commerces à la pointe de la technologie.
- **Évolutivité** : L'architecture modulaire Spring Boot/Angular permet l'ajout facile de nouvelles fonctionnalités (gestion des commandes, statistiques avancées, etc.).

CONCLUSION GÉNÉRALE

1. Synthèse du travail réalisé

Le présent mémoire a documenté l'intégralité du processus de conception et de réalisation d'une plateforme web pour la gestion multi-quincailleries, depuis l'analyse des besoins jusqu'à la phase de validation.

Le projet a réussi à matérialiser une solution concrète à la problématique de la gestion dispersée dans le commerce de quincaillerie. En s'appuyant sur une architecture moderne **Client-Serveur** avec une **API RESTful Spring Boot** et un **Frontend Angular**, nous avons construit un outil à la fois puissant, sécurisé et convivial. L'adoption de technologies d'entreprise telles que Spring Boot 3.5.5, Spring Security avec JWT, Angular 20 et PostgreSQL a non seulement assuré la robustesse et la performance de la solution, mais a également permis une application concrète des concepts avancés du Génie Logiciel.

Les phases de modélisation UML (Cas d'Utilisation, Classes, Séquence) ont permis de garantir la cohérence du système avant le codage, et la stratégie de tests rigoureuse (unitaires, intégration, sécurité) a validé sa conformité aux exigences fonctionnelles et non fonctionnelles.

2. Objectifs atteints

Tous les objectifs fixés en introduction ont été intégralement atteints :

- L'analyse des besoins et la modélisation UML sont complètes et ont servi de guide fiable.
- Un système d'authentification robuste (JWT) est opérationnel, sécurisant toutes les routes de gestion.
- Les fonctionnalités **CRUD** de gestion des magasins et des produits sont implémentées et fonctionnent conformément au modèle de propriété (un utilisateur gère ses magasins).
- L'interface utilisateur Angular offre un *Dashboard* d'administration et une **vitrine publique** responsive.
- Le système a été validé par des tests complets, assurant sa stabilité.

3. Apports personnels et professionnels

Ce projet de Licence en Génie Logiciel constitue une expérience professionnelle formatrice majeure.

- **Maîtrise Technique** : Il a permis d'acquérir une expertise approfondie dans la **Stack Full-Stack Spring/Angular**, une compétence très recherchée dans l'industrie.

- **Gestion de Projet** : Il a renforcé la capacité à planifier, structurer et exécuter un projet complexe en respectant une méthodologie rigoureuse.
- **Architecture Logicielle** : Compréhension et mise en œuvre pratique des principes d'architecture distribuée, de la sécurité par token (JWT) et de la séparation des préoccupations (DTOs, Repository Pattern).

4. Limites de la solution

Bien que fonctionnelle et robuste, la solution actuelle présente certaines limites qui ouvrent la voie à des améliorations futures :

- **Gestion Avancée des Stocks** : Le système gère la quantité en stock, mais n'inclut pas de fonctionnalités avancées comme la gestion des entrées/sorties de caisse, les alertes de stock minimal ou la gestion des fournisseurs.
- **Fonctionnalités Utilisateur** : L'application ne gère actuellement qu'un seul rôle d'utilisateur (le Propriétaire) et le Visiteur. Une gestion fine des rôles (Vendeur, Super-Administrateur) pourrait être nécessaire.
- **Paiement et Commande** : Aucune fonctionnalité transactionnelle (panier, paiement en ligne) n'est intégrée.

5. Perspectives et améliorations futures

Les perspectives d'évolution de la plateforme sont nombreuses et prometteuses :

1. **Extension Fonctionnelle (Ventes)** : Intégration d'un module de caisse (Point-of-Sale - POS) pour enregistrer les ventes et les retraits directement sur le stock.
2. **Statistiques et Reporting** : Ajout de tableaux de bord analytiques pour les propriétaires (meilleures ventes, stocks faibles, chiffre d'affaires par magasin).
3. **Amélioration de la Sécurité/Rôles** : Mise en place d'un système de rôles/permissions pour permettre à un propriétaire de déléguer la gestion de ses magasins à des employés.
4. **Évolution Mobile** : Développement d'une application mobile dédiée (via Flutter, React Native ou même PWA) pour un accès optimisé aux fonctionnalités clés.

Ce projet marque l'achèvement d'un cycle d'apprentissage et le point de départ d'une carrière en Génie Logiciel. La plateforme **[Nom de votre application]** est un témoignage tangible de la capacité à transformer une problématique métier en une solution logicielle innovante, structurée et sécurisée.

BIBLIOGRAPHIE / WEBOGRAPHIE

1. Ouvrages et Manuels

1. **FOWLER, Martin.** *UML Distilled: A Brief Guide to the Standard Object Modeling Language*. Addison-Wesley Professional, 3ème édition, 2004.
2. **GAMMA, Erich, et al.** *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley Professional, 1994.
3. **BLOCH, Joshua.** *Effective Java*. Addison-Wesley Professional, 3ème édition, 2018.

2. Articles et Publications

4. **FIELDING, Roy Thomas.** *Architectural Styles and the Design of Network-based Software Architectures*. Ph.D. Dissertation, University of California, Irvine, 2000 (Définition de REST).

3. Ressources Web et Documentation Technique

5. **Documentation Officielle Spring Boot.** *Spring Boot Reference Documentation*. <https://docs.spring.io/spring-boot/documentation.html>
6. **Documentation Officielle Angular.** *Angular Documentation*. <https://angular.io/docs>
7. **Documentation Officielle PostgreSQL.** *PostgreSQL Documentation*. <https://www.postgresql.org/docs/>
8. **Site Officiel JWT.** *Introduction to JSON Web Tokens*. <https://jwt.io/introduction>
9. **Ressources sur les Patrons de Conception.** <https://refactoring.guru/fr/design-patterns>